

CIS 6270

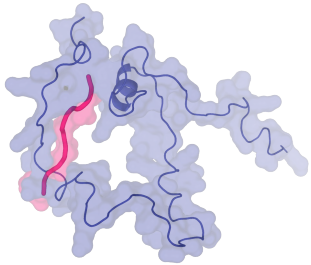
Module 5: Discrete Flow Matching

Lecture 5.3: Dirichlet, Fisher-Rao, and Gumbel-Softmax Flows

Tue, Nov 3, 2026

Pranam Chatterjee, PhD
Assistant Professor of Bioengineering
Assistant Professor of Computer and Information Science
University of Pennsylvania

Penn Engineering
University of Pennsylvania



Put a distance on the tokens

Put a distance on the tokens

Choose a distance d between tokens and concentrate on x_1:

$$p_t(y \mid x_1) = \mathrm{softmax}_y\big(-\beta_t\, d(y, x_1)\big), \qquad \beta_0 = 0,\ \beta_1 = \infty$$

Put a distance on the tokens

Choose a distance d between tokens and concentrate on x_1:

$$\boxed{p_t(y \mid x_1) = \mathrm{softmax}_y\big(-\beta_t\, d(y, x_1)\big), \qquad \beta_0 = 0,\ \beta_1 = \infty}$$

The conditional probability of the token y is a softmax over tokens of negative β_t times the distance $d(y, x_1)$ from y to the data token, where β_t grows from zero at t = 0 to infinity at t = 1.

Put a distance on the tokens

Choose a distance d between tokens and concentrate on x_1:

$$\boxed{p_t(y \mid x_1) = \mathrm{softmax}_y\big(-\beta_t\, d(y, x_1)\big), \qquad \beta_0 = 0,\ \beta_1 = \infty}$$

The conditional probability of the token y is a softmax over tokens of negative β_t times the distance $d(y, x_1)$ from y to the data token, where β_t grows from zero at t = 0 to infinity at t = 1.

$$u_t^\star(y, x \mid x_1) = p_t(y \mid x_1)\ \dot\beta_t \left[\, d(x, x_1) - d(y, x_1) \,\right]_+$$

Put a distance on the tokens

Choose a distance d between tokens and concentrate on x_1:

$$\boxed{p_t(y \mid x_1) = \mathrm{softmax}_y\big(-\beta_t\, d(y, x_1)\big), \qquad \beta_0 = 0,\ \beta_1 = \infty}$$

The conditional probability of the token y is a softmax over tokens of negative β_t times the distance $d(y, x_1)$ from y to the data token, where β_t grows from zero at t = 0 to infinity at t = 1.

$$u_t^\star(y, x \mid x_1) = p_t(y \mid x_1)\ \dot\beta_t \left[\, d(x, x_1) - d(y, x_1) \,\right]_+$$

The kinetic-optimal velocity from x to y equals the probability of y times the rate $\dot\beta_t$ times the positive part of the distance from x to the data token minus the distance from y to the data token.

Put a distance on the tokens

Choose a distance d between tokens and concentrate on x_1:

$$\boxed{p_t(y \mid x_1) = \mathrm{softmax}_y\big(-\beta_t\, d(y, x_1)\big), \qquad \beta_0 = 0,\ \beta_1 = \infty}$$

The conditional probability of the token y is a softmax over tokens of negative β_t times the distance $d(y, x_1)$ from y to the data token, where β_t grows from zero at t = 0 to infinity at t = 1.

$$u_t^\star(y, x \mid x_1) = p_t(y \mid x_1)\ \dot\beta_t \left[\, d(x, x_1) - d(y, x_1) \,\right]_+$$

The kinetic-optimal velocity from x to y equals the probability of y times the rate $\dot\beta_t$ times the positive part of the distance from x to the data token minus the distance from y to the data token.

Probability only moves to tokens strictly closer to x_1, faster when the step in distance is larger.

Let’s build a metric path for DNA

Let’s build a metric path for DNA

Transitions swap a purine for a purine, A and G, or a pyrimidine for a pyrimidine, C and T. They are the more common mutations.

Let’s build a metric path for DNA

Transitions swap a purine for a purine, A and G, or a pyrimidine for a pyrimidine, C and T. They are the more common mutations.

Let d = 1 for a transition and d = 2 for a transversion. With x_1 = G, d(·, G) = (1, 2, 0, 2) for (A, C, G, T).

Let’s build a metric path for DNA

Transitions swap a purine for a purine, A and G, or a pyrimidine for a pyrimidine, C and T. They are the more common mutations.

Let d = 1 for a transition and d = 2 for a transversion. With x_1 = G, d(·, G) = (1, 2, 0, 2) for (A, C, G, T).

At t = 0.5, with $\beta_t = t/(1 - t)$,

$$\beta = 1 : \qquad p_t(\cdot \mid \mathrm{G}) = (0.2245,\ 0.0826,\ 0.6103,\ 0.0826)$$

Let’s build a metric path for DNA

Transitions swap a purine for a purine, A and G, or a pyrimidine for a pyrimidine, C and T. They are the more common mutations.

Let d = 1 for a transition and d = 2 for a transversion. With x_1 = G, d(·, G) = (1, 2, 0, 2) for (A, C, G, T).

At t = 0.5, with $\beta_t = t/(1 - t)$,

$$\beta = 1: \qquad p_t(\cdot \mid \mathrm{G}) = (0.2245,\ 0.0826,\ 0.6103,\ 0.0826)$$

With β = 1, the softmax of negative distances gives probabilities 0.2245 for A, 0.0826 for C, 0.6103 for G, and 0.0826 for T.

Let’s build a metric path for DNA

Transitions swap a purine for a purine, A and G, or a pyrimidine for a pyrimidine, C and T. They are the more common mutations.

Let d = 1 for a transition and d = 2 for a transversion. With x_1 = G, d(·, G) = (1, 2, 0, 2) for (A, C, G, T).

At t = 0.5, with $\beta_t = t/(1 - t)$,

$$\beta = 1: \qquad p_t(\cdot \mid \mathrm{G}) = (0.2245,\ 0.0826,\ 0.6103,\ 0.0826)$$

With $\beta = 4$, a C moves toward A and toward G:

$$u_t^\star(\mathrm{A}, \mathrm{C}) = 0.2245 \times 4 \times (2 - 1) = 0.898, \qquad u_t^\star(\mathrm{G}, \mathrm{C}) = 0.6103 \times 4 \times 2 = 4.882$$

Let’s build a metric path for DNA

Transitions swap a purine for a purine, A and G, or a pyrimidine for a pyrimidine, C and T. They are the more common mutations.

Let d = 1 for a transition and d = 2 for a transversion. With x_1 = G, d(·, G) = (1, 2, 0, 2) for (A, C, G, T).

At t = 0.5, with $\beta_t = t/(1 - t)$,

$$\beta = 1: \qquad p_t(\cdot \mid \mathrm{G}) = (0.2245,\ 0.0826,\ 0.6103,\ 0.0826)$$

With $\beta = 4$, a C moves toward A and toward G:

$$u_t^\star(\mathrm{A}, \mathrm{C}) = 0.2245 \times 4 \times (2 - 1) = 0.898, \qquad u_t^\star(\mathrm{G}, \mathrm{C}) = 0.6103 \times 4 \times 2 = 4.882$$

The rate from C to A is 0.2245 times 4 times the distance gain of one, or 0.898, and the rate from C to G is 0.6103 times 4 times the gain of two, or 4.882.

Let’s build a metric path for DNA

Transitions swap a purine for a purine, A and G, or a pyrimidine for a pyrimidine, C and T. They are the more common mutations.

Let d = 1 for a transition and d = 2 for a transversion. With x_1 = G, d(·, G) = (1, 2, 0, 2) for (A, C, G, T).

At t = 0.5, with $\beta_t = t/(1-t)$,

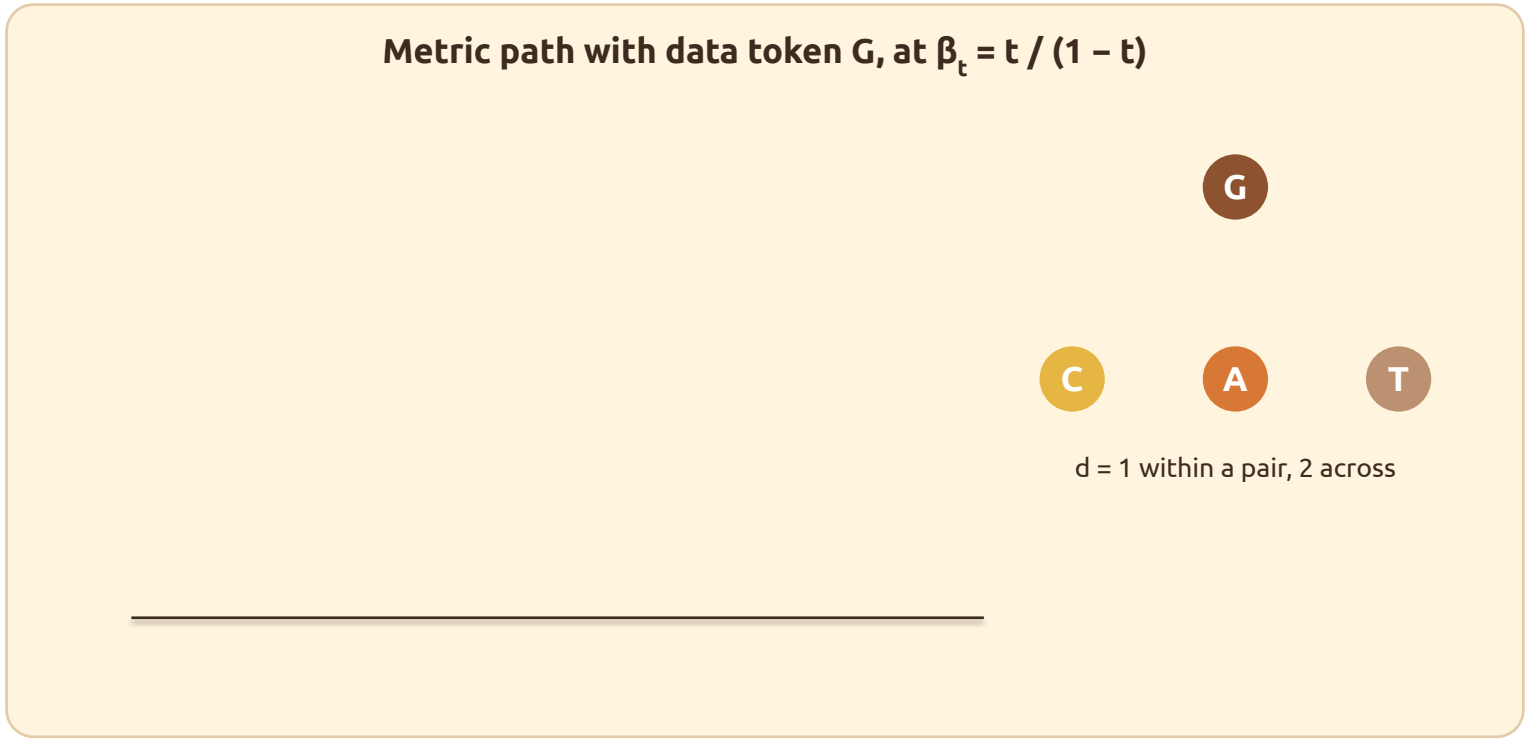
$$\beta = 1: \qquad p_t(\cdot \mid \mathrm{G}) = (0.2245,\ 0.0826,\ 0.6103,\ 0.0826)$$

With $\beta = 4$, a C moves toward A and toward G:

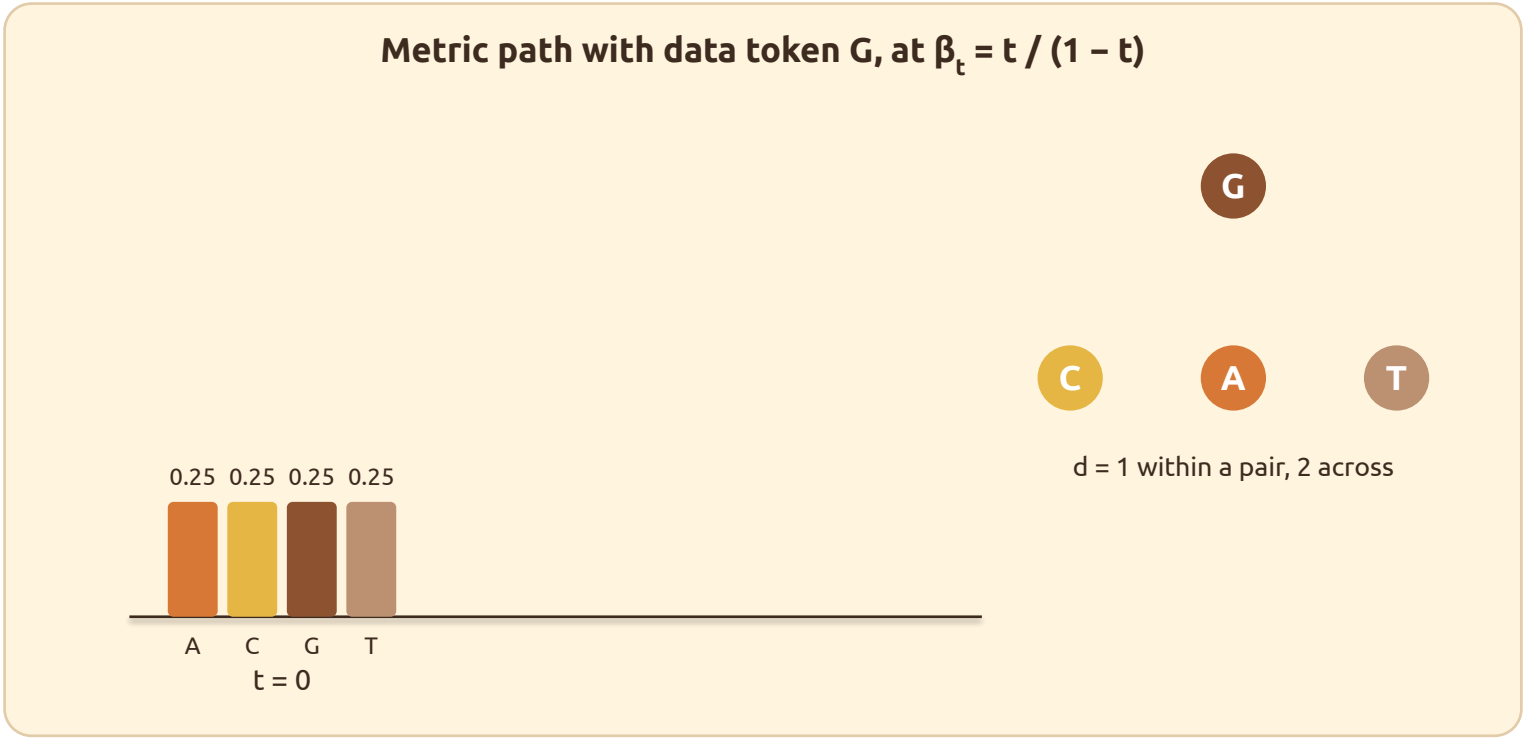
$$u_t^\star(\mathrm{A}, \mathrm{C}) = 0.2245 \times 4 \times (2-1) = 0.898, \qquad u_t^\star(\mathrm{G}, \mathrm{C}) = 0.6103 \times 4 \times 2 = 4.882$$

A pyrimidine can reach G directly or through the purine A. We use the token distances to distinguish smaller changes from larger ones.

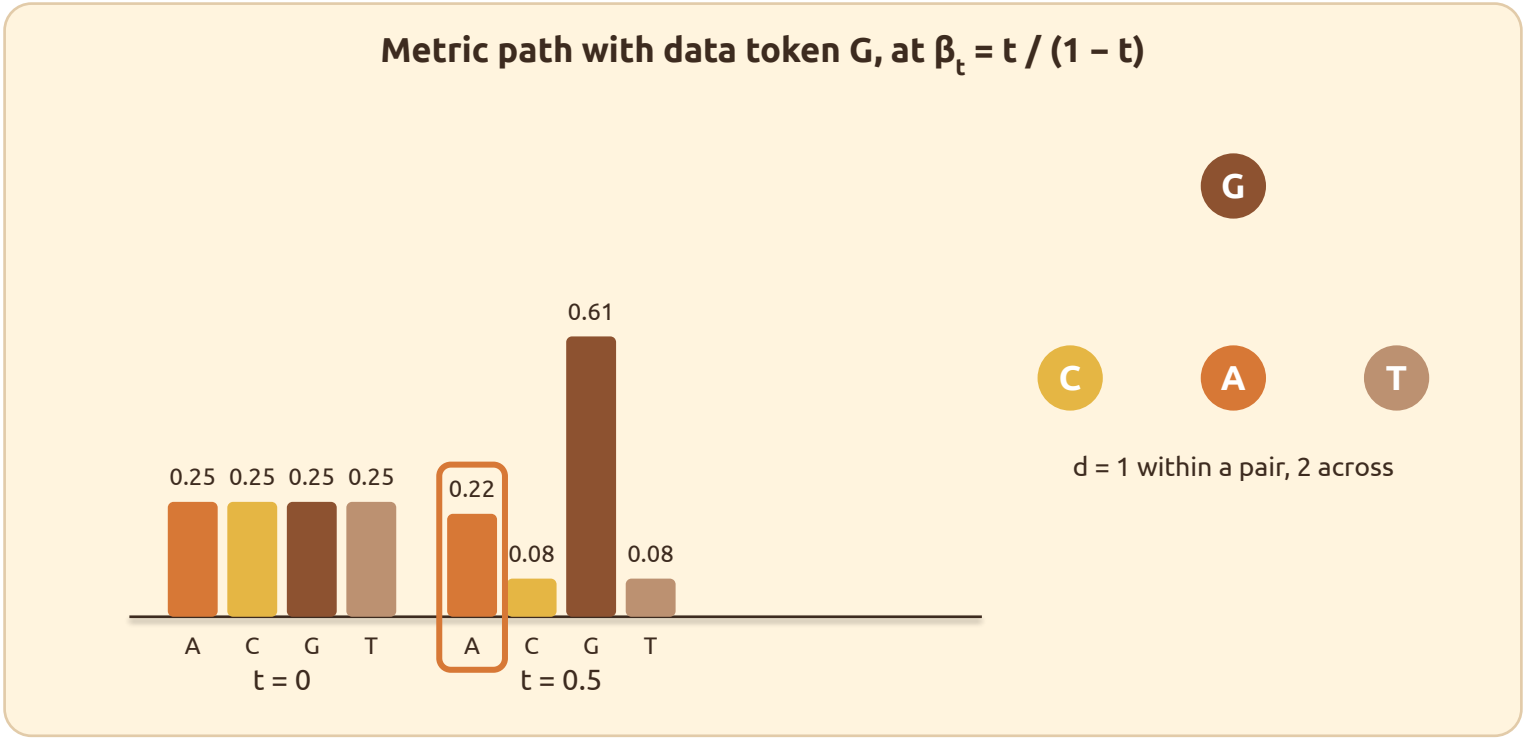
Now watch the metric path concentrate on G



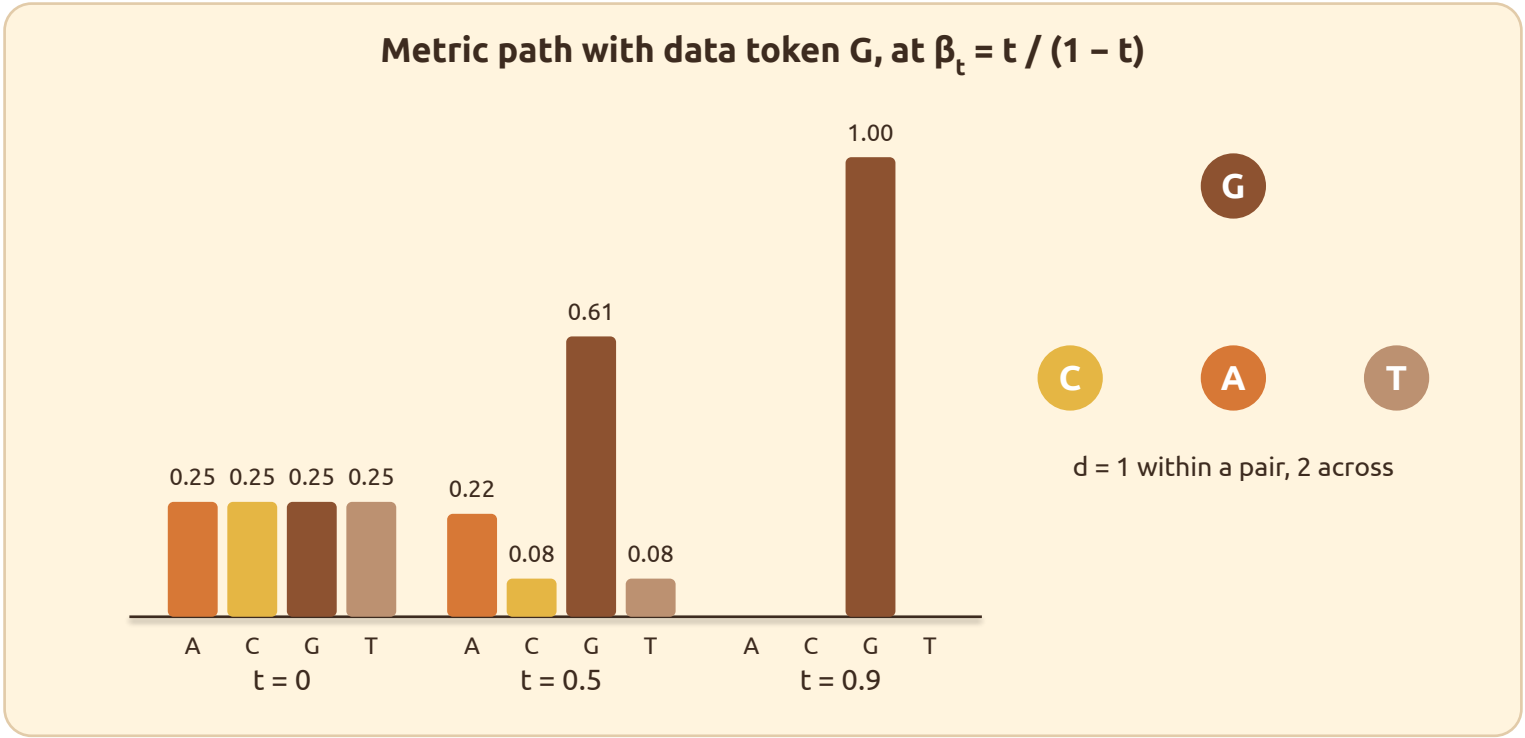
At t = 0, every base is equally likely



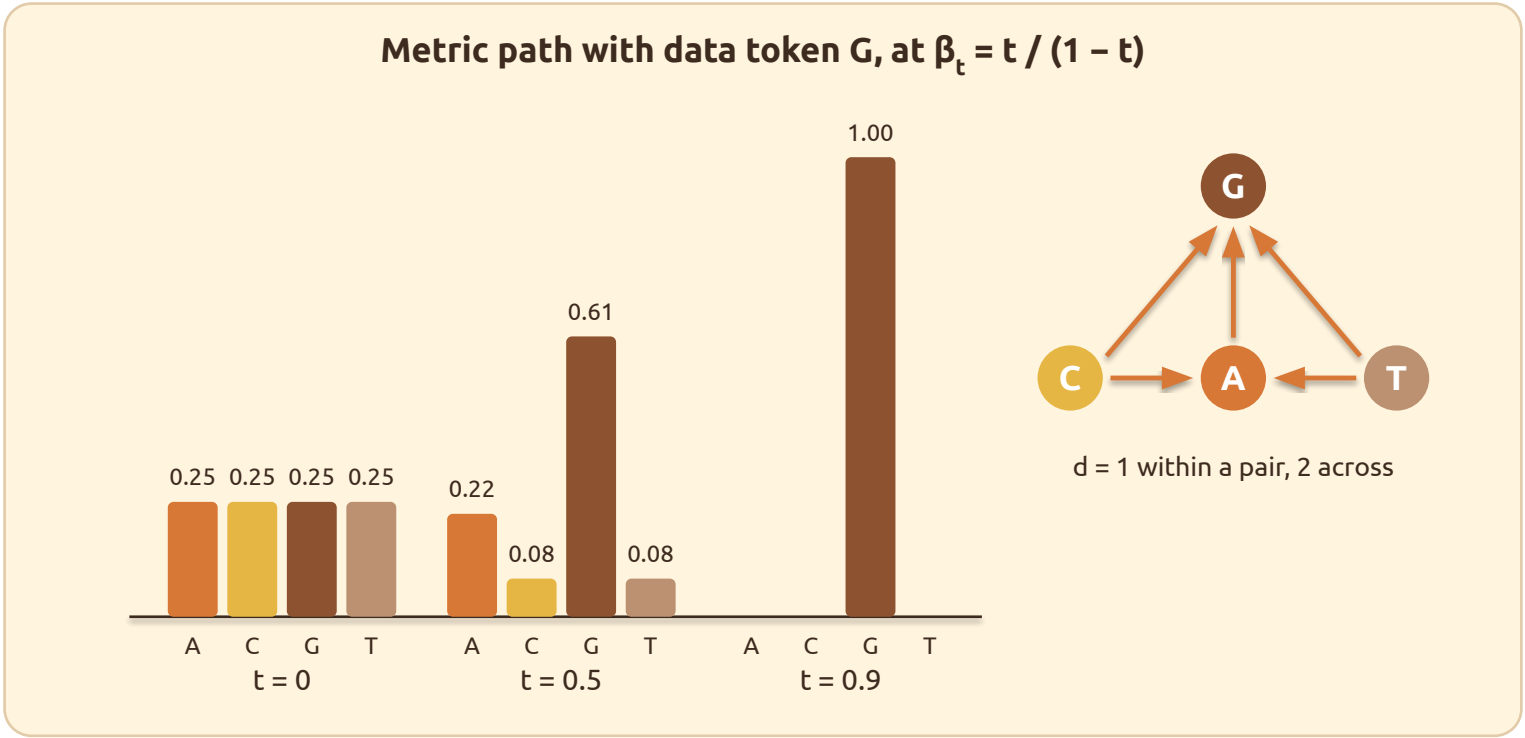
At t = 0.5, A has more probability than C or T because it is one transition from G



At t = 0.9, nearly all probability is on G



The arrows show the only allowed flows: toward bases closer to G



Now minimize the energy over the path as well

Now minimize the energy over the path as well

In the same paper, they also minimize this energy over the path itself, with fixed endpoints p and q.

Now minimize the energy over the path as well

In the same paper, they also minimize this energy over the path itself, with fixed endpoints p and q.

At the optimal velocity, the energy only depends on $\sqrt{p_t}$:

$$\int_0^1 \sum_x \frac{\dot{p}_t(x)^2}{p_t(x)}\,dt = 4\int_0^1 \sum_x \left(\frac{d}{dt}\sqrt{p_t(x)}\right)^2 dt$$

Now minimize the energy over the path as well

In the same paper, they also minimize this energy over the path itself, with fixed endpoints p and q.

At the optimal velocity, the energy only depends on $\sqrt{p_t}$:

$$\int_0^1 \sum_x \frac{\dot{p}_t(x)^2}{p_t(x)}\,dt = 4\int_0^1 \sum_x \left(\frac{d}{dt}\sqrt{p_t(x)}\right)^2 dt$$

The energy, the integral over time of the sum over states of the squared rate of change of $p_t(x)$ divided by $p_t(x)$, equals four times the integral of the squared speed of the square-root vector $\sqrt{p_t}$.

Now minimize the energy over the path as well

In the same paper, they also minimize this energy over the path itself, with fixed endpoints p and q.

At the optimal velocity, the energy only depends on $\sqrt{p_t}$:

$$\int_0^1 \sum_x \frac{\dot{p}_t(x)^2}{p_t(x)}\,dt = 4\int_0^1 \sum_x \left(\frac{d}{dt}\sqrt{p_t(x)}\right)^2 dt$$

The energy, the integral over time of the sum over states of the squared rate of change of $p_t(x)$ divided by $p_t(x)$, equals four times the integral of the squared speed of the square-root vector $\sqrt{p_t}$.

The vector $\sqrt{p_t}$ has unit length, so it lies on a sphere. The shortest path between two points on that sphere is a great-circle arc.

Now minimize the energy over the path as well

In the same paper, they also minimize this energy over the path itself, with fixed endpoints p and q.

At the optimal velocity, the energy only depends on $\sqrt{p_t}$:

$$\int_0^1 \sum_x \frac{\dot{p}_t(x)^2}{p_t(x)}\,dt = 4\int_0^1 \sum_x \left(\frac{d}{dt}\sqrt{p_t(x)}\right)^2 dt$$

The energy, the integral over time of the sum over states of the squared rate of change of $p_t(x)$ divided by $p_t(x)$, equals four times the integral of the squared speed of the square-root vector $\sqrt{p_t}$.

The least-energy probability path follows the shortest arc between its square-root endpoints.

Read the schedule off that arc

Read the schedule off that arc

For a point-mass target, the arc is a mixture path with a token-dependent schedule:

$$\kappa_t(x_1) = 1 - \frac{\sin^2\big((1-t)\,\Omega\big)}{\sin^2\Omega}, \qquad \Omega = \arccos\sqrt{p(x_1)}$$

Read the schedule off that arc

For a point-mass target, the arc is a mixture path with a token-dependent schedule:

$$\kappa_t(x_1) = 1 - \frac{\sin^2\big((1-t)\,\Omega\big)}{\sin^2\Omega}, \qquad \Omega = \arccos\sqrt{p(x_1)}$$

The schedule $\kappa_t(x_1)$ equals one minus the squared sine of one minus t times Ω, divided by the squared sine of Ω, where the angle Ω is the arccosine of the square root of the source probability of the data token x_1.

Read the schedule off that arc

For a point-mass target, the arc is a mixture path with a token-dependent schedule:

$$\kappa_t(x_1) = 1 - \frac{\sin^2\big((1-t)\,\Omega\big)}{\sin^2\Omega}, \qquad \Omega = \arccos\sqrt{p(x_1)}$$

The schedule $\kappa_t(x_1)$ equals one minus the squared sine of one minus t times Ω, divided by the squared sine of Ω, where the angle Ω is the arccosine of the square root of the source probability of the data token x_1.

A token that is already likely under the source starts with a small angle, so its schedule rises early.

Read the schedule off that arc

For a point-mass target, the arc is a mixture path with a token-dependent schedule:

$$\kappa_t(x_1) = 1 - \frac{\sin^2\big((1-t)\,\Omega\big)}{\sin^2\Omega}, \qquad \Omega = \arccos\sqrt{p(x_1)}$$

The schedule $\kappa_t(x_1)$ equals one minus the squared sine of one minus t times Ω, divided by the squared sine of Ω, where the angle Ω is the arccosine of the square root of the source probability of the data token x_1.

A token that is already likely under the source starts with a small angle, so its schedule rises early.

For the mask source, $p(x_1) = 0$ gives $\Omega = \pi/2$ and $\kappa_t = \sin^2(\pi t/2)$.

Next, we look for a likelihood bound

Next, we look for a likelihood bound

In the same paper, they derive an evidence lower bound for any path and velocity, with a closed form for mixture paths.

Next, we look for a likelihood bound

In the same paper, they derive an evidence lower bound for any path and velocity, with a closed form for mixture paths.

For the mask source with one schedule for every token, it reduces to

$$\log p_1^\theta(x_1) \;\geq\; \int_0^1 \frac{\dot{\kappa}_t}{1-\kappa_t}\, \mathbb{E}_{x_t}\Big[\sum_{i:\, x_t^i = m} \log p_{1|t}^\theta(x_1^i \mid x_t) \Big]\, dt$$

Next, we look for a likelihood bound

In the same paper, they derive an evidence lower bound for any path and velocity, with a closed form for mixture paths.

For the mask source with one schedule for every token, it reduces to

$$\log p_1^\theta(x_1) \;\geq\; \int_0^1 \frac{\dot{\kappa}_t}{1-\kappa_t}\, \mathbb{E}_{x_t}\Big[\sum_{i:\, x_t^i = m} \log p_{1|t}^\theta(x_1^i \mid x_t) \Big]\, dt$$

The model's log-likelihood of the data sequence x_1 is at least the integral over time of $\dot{\kappa}_t$ divided by one minus κ_t, times the expectation over the noisy sequence x_t of the sum over masked positions of the log probability the denoiser $p^\theta_{1|t}$ assigns to the true data token.

Next, we look for a likelihood bound

In the same paper, they derive an evidence lower bound for any path and velocity, with a closed form for mixture paths.

For the mask source with one schedule for every token, it reduces to

$$\log p_1^\theta(x_1) \;\geq\; \int_0^1 \frac{\dot{\kappa}_t}{1-\kappa_t}\, \mathbb{E}_{x_t}\Big[\sum_{i:\, x_t^i = m} \log p_{1|t}^\theta(x_1^i \mid x_t) \Big]\, dt$$

The model's log-likelihood of the data sequence x_1 is at least the integral over time of $\dot{\kappa}_t$ divided by one minus κ_t, times the expectation over the noisy sequence x_t of the sum over masked positions of the log probability the denoiser $p^\theta_{1|t}$ assigns to the true data token.

We recover MDLM's likelihood bound using weighted cross-entropy.

Next, we look for a likelihood bound

In the same paper, they derive an evidence lower bound for any path and velocity, with a closed form for mixture paths.

For the mask source with one schedule for every token, it reduces to

$$\log p_1^\theta(x_1) \;\geq\; \int_0^1 \frac{\dot{\kappa}_t}{1-\kappa_t}\, \mathbb{E}_{x_t}\Big[\sum_{i:\, x_t^i = m} \log p_{1|t}^\theta(x_1^i \mid x_t) \Big]\, dt$$

The model's log-likelihood of the data sequence x_1 is at least the integral over time of $\dot{\kappa}_t$ divided by one minus κ_t, times the expectation over the noisy sequence x_t of the sum over masked positions of the log probability the denoiser $p^\theta_{1|t}$ assigns to the true data token.

We recover MDLM's likelihood bound using weighted cross-entropy. Unweighted cross-entropy has the same minimizer.

Take a single masked base, with the linear schedule

Take a single masked base, with the linear schedule

For a single masked base with $\kappa_t = t$, the weight is $1/(1 - t)$, while the masking probability is $1 - t$.

Take a single masked base, with the linear schedule

For a single masked base with $\kappa_t = t$, the weight is $1/(1 - t)$, while the masking probability is $1 - t$.

Their product is 1, so the integral of a constant log $p^{\theta}(G)$ over $[0, 1]$ is that log probability itself.

Take a single masked base, with the linear schedule

For a single masked base with $\kappa_t = t$, the weight is $1/(1 - t)$, while the masking probability is $1 - t$.

Their product is 1, so the integral of a constant log $p^\theta(G)$ over $[0, 1]$ is that log probability itself.

With $p^\theta(G) = 0.6$, the bound comes to $\log 0.6 = -0.511$, the negative cross-entropy of that one position.

Kinetic-optimal flow on A and G

Toy masses on A and G are (0.8, 0.2), with derivatives (−0.1, +0.1). Use the kinetic-optimal rates for the reciprocal-probability edge weights.

Kinetic-optimal flow on A and G

Toy masses on A and G are (0.8, 0.2), with derivatives (−0.1, +0.1). Use the kinetic-optimal rates for the reciprocal-probability edge weights.

$$u^*(\mathrm{G},\mathrm{A}) = 0.2\left[\frac{0.1}{0.2} - \frac{-0.1}{0.8}\right]_+ = 0.125$$

Kinetic-optimal flow on A and G

Toy masses on A and G are (0.8, 0.2), with derivatives (−0.1, +0.1). Use the kinetic-optimal rates for the reciprocal-probability edge weights.

$$u^*(\mathrm{G},\mathrm{A}) = 0.2\left[\frac{0.1}{0.2} - \frac{-0.1}{0.8}\right]_+ = 0.125$$

G grows at relative rate 0.5 while A shrinks at relative rate 0.125. Their difference sets the jump rate.

Kinetic-optimal flow on A and G

Toy masses on A and G are (0.8, 0.2), with derivatives (−0.1, +0.1). Use the kinetic-optimal rates for the reciprocal-probability edge weights.

$$u^*(\mathrm{G},\mathrm{A}) = 0.2\left[\frac{0.1}{0.2} - \frac{-0.1}{0.8}\right]_+ = 0.125$$

G grows at relative rate 0.5 while A shrinks at relative rate 0.125. Their difference sets the jump rate.

$$j(\mathrm{G},\mathrm{A}) = (0.8)(0.125) = 0.1, \qquad p_{t+0.1}(\mathrm{G}) \approx 0.2 + 0.1(0.1) = 0.21$$

Kinetic-optimal flow on A and G

Toy masses on A and G are (0.8, 0.2), with derivatives (−0.1, +0.1). Use the kinetic-optimal rates for the reciprocal-probability edge weights.

$$u^*(\mathrm{G},\mathrm{A}) = 0.2\left[\frac{0.1}{0.2} - \frac{-0.1}{0.8}\right]_+ = 0.125$$

G grows at relative rate 0.5 while A shrinks at relative rate 0.125. Their difference sets the jump rate.

$$j(\mathrm{G},\mathrm{A}) = (0.8)(0.125) = 0.1, \qquad p_{t+0.1}(\mathrm{G}) \approx 0.2 + 0.1(0.1) = 0.21$$

The flux is 0.1. A short first-order update moves the G mass from 0.20 to 0.21.

Kinetic-optimal flow on A and G

Toy masses on A and G are (0.8, 0.2), with derivatives (−0.1, +0.1). Use the kinetic-optimal rates for the reciprocal-probability edge weights.

$$u^*(\mathrm{G},\mathrm{A}) = 0.2\left[\frac{0.1}{0.2} - \frac{-0.1}{0.8}\right]_+ = 0.125$$

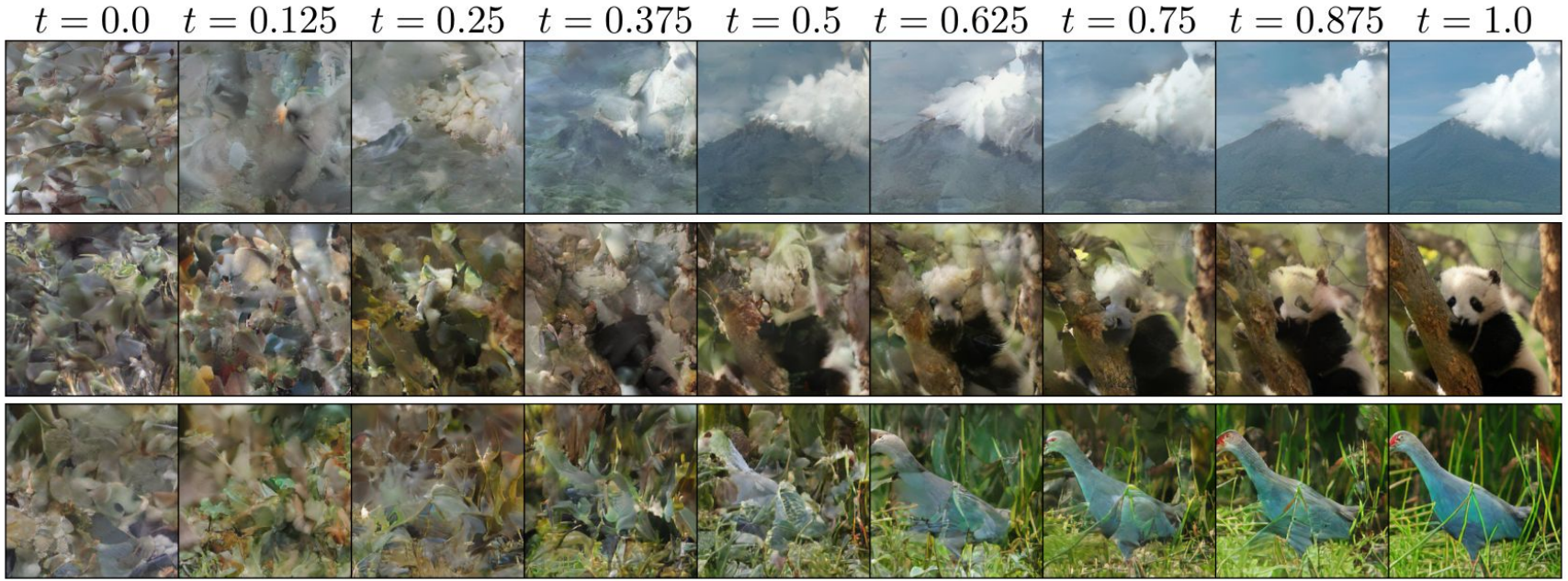
G grows at relative rate 0.5 while A shrinks at relative rate 0.125. Their difference sets the jump rate.

$$j(\mathrm{G},\mathrm{A}) = (0.8)(0.125) = 0.1, \qquad p_{t+0.1}(\mathrm{G}) \approx 0.2 + 0.1(0.1) = 0.21$$

The flux is 0.1. A short first-order update moves the G mass from 0.20 to 0.21.

The rate describes one particle. The flux tells us how much total probability moves.

On image tokens, a metric path runs from noise to an image



Their metric-induced path on image tokens, from t = 0 to t = 1.

Shaul,..,Chen. *ICLR* (2025)

Now look at the FID numbers in Table 3 of the paper

Table 3: Face-blurred ImageNet-256 with the Llama-B architecture (111M parameters).
* denotes our reimplementation.

Method	NFE	FID
LlamaGen (AR) (Sun et al., 2024)	256	5.46
LlamaGen (AR)* (Sun et al., 2024)	256	4.81
DFM - Mask* (Gat et al., 2024)	100	5.72
DFM - Metric (Ours)	100	**4.50**

Shaul,..,Chen. *ICLR* (2025)

On face-blurred ImageNet at 256 by 256, the masked path reaches an FID of 5.72

Table 3: Face-blurred ImageNet-256 with the Llama-B architecture (111M parameters).
* denotes our reimplementation.

Method	NFE	FID
LlamaGen (AR) (Sun et al., 2024)	256	5.46
LlamaGen (AR)* (Sun et al., 2024)	256	4.81
DFM - Mask* (Gat et al., 2024)	100	5.72
DFM - Metric (Ours)	100	**4.50**

Shaul,..,Chen. *ICLR* (2025)

With the metric path, FID improves to 4.50 in 100 steps

Table 3: Face-blurred ImageNet-256 with the Llama-B architecture (111M parameters).
* denotes our reimplementation.

Method	NFE	FID
LlamaGen (AR) (Sun et al., 2024)	256	5.46
LlamaGen (AR)* (Sun et al., 2024)	256	4.81
DFM - Mask* (Gat et al., 2024)	100	5.72
DFM - Metric (Ours)	100	**4.50**

Shaul,..,Chen. *ICLR* (2025)

The autoregressive LlamaGen reaches 4.81 in 256 steps

Table 3: Face-blurred ImageNet-256 with the Llama-B architecture (111M parameters).
* denotes our reimplementation.

Method	NFE	FID
LlamaGen (AR) (Sun et al., 2024)	256	5.46
LlamaGen (AR)* (Sun et al., 2024)	256	4.81
DFM - Mask* (Gat et al., 2024)	100	5.72
DFM - Metric (Ours)	100	**4.50**

Shaul,..,Chen. *ICLR* (2025)

We choose the probability path just as we chose the corruption process in discrete diffusion

Table 3: Face-blurred ImageNet-256 with the Llama-B architecture (111M parameters).
* denotes our reimplementation.

Method	NFE	FID
LlamaGen (AR) (Sun et al., 2024)	256	5.46
LlamaGen (AR)* (Sun et al., 2024)	256	4.81
DFM - Mask* (Gat et al., 2024)	100	5.72
DFM - Metric (Ours)	100	**4.50**

Shaul,..,Chen. *ICLR* (2025)

Could the flow insert and delete tokens so the sequence length can change too?

Edit Flows

Edit Flows: Variable Length Discrete Flow Matching with Sequence-Level Edit Operations

Marton Havasi FAIR at Meta
Brian Karrer FAIR at Meta
Itai Gat FAIR at Meta
Ricky T. Q. Chen FAIR at Meta

Havasi,..,Chen. *NeurIPS* (2025)

So far, every path has had a fixed length

So far, every path has had a fixed length

A substitution changes a token, but it never adds or removes one.

So far, every path has had a fixed length

A substitution changes a token, but it never adds or removes one.

If only substitutions have positive rates, the length never changes, so p_t stays on sequences of one length for all t.

So far, every path has had a fixed length

A substitution changes a token, but it never adds or removes one.

If only substitutions have positive rates, the length never changes, so p_t stays on sequences of one length for all t.

DNA, proteins, molecules, and code vary in length. With a mask model, we must choose the length before sampling.

In Edit Flows we also allow insertions and deletions

In Edit Flows we also allow insertions and deletions

In Edit Flows, there are three edits at each position i:

$$\mathrm{ins}(x, i, a), \qquad \mathrm{del}(x, i), \qquad \mathrm{sub}(x, i, a)$$

In Edit Flows we also allow insertions and deletions

In Edit Flows, there are three edits at each position i:

$$\mathrm{ins}(x, i, a), \qquad \mathrm{del}(x, i), \qquad \mathrm{sub}(x, i, a)$$

An insertion places the token a to the right of position i, a deletion removes the token at position i, and a substitution replaces the token at position i with a.

In Edit Flows we also allow insertions and deletions

In Edit Flows, there are three edits at each position i:

$$\mathrm{ins}(x, i, a), \qquad \mathrm{del}(x, i), \qquad \mathrm{sub}(x, i, a)$$

An insertion places the token a to the right of position i, a deletion removes the token at position i, and a substitution replaces the token at position i with a.

$$u_t(\mathrm{ins}(x,i,a) \mid x) = \lambda_{t,i}^{\mathrm{ins}}(x)\, Q_{t,i}^{\mathrm{ins}}(a \mid x), \quad u_t(\mathrm{del}(x,i) \mid x) = \lambda_{t,i}^{\mathrm{del}}(x), \quad u_t(\mathrm{sub}(x,i,a) \mid x) = \lambda_{t,i}^{\mathrm{sub}}(x)\, Q_{t,i}^{\mathrm{sub}}(a \mid x)$$

In Edit Flows we also allow insertions and deletions

In Edit Flows, there are three edits at each position i:

$$\mathrm{ins}(x, i, a), \qquad \mathrm{del}(x, i), \qquad \mathrm{sub}(x, i, a)$$

An insertion places the token a to the right of position i, a deletion removes the token at position i, and a substitution replaces the token at position i with a.

$$u_t(\mathrm{ins}(x,i,a) \mid x) = \lambda^{\mathrm{ins}}_{t,i}(x)\, Q^{\mathrm{ins}}_{t,i}(a \mid x), \quad u_t(\mathrm{del}(x,i) \mid x) = \lambda^{\mathrm{del}}_{t,i}(x), \quad u_t(\mathrm{sub}(x,i,a) \mid x) = \lambda^{\mathrm{sub}}_{t,i}(x)\, Q^{\mathrm{sub}}_{t,i}(a \mid x)$$

Each insertion rate is an intensity $\lambda^{\mathrm{ins}}_{t,i}(x)$ times a token distribution $Q^{\mathrm{ins}}_{t,i}(a \mid x)$, each deletion rate is an intensity $\lambda^{\mathrm{del}}_{t,i}(x)$, and each substitution rate is an intensity $\lambda^{\mathrm{sub}}_{t,i}(x)$ times a token distribution $Q^{\mathrm{sub}}_{t,i}(a \mid x)$.

In Edit Flows we also allow insertions and deletions

In Edit Flows, there are three edits at each position i:

$$\mathrm{ins}(x, i, a), \qquad \mathrm{del}(x, i), \qquad \mathrm{sub}(x, i, a)$$

An insertion places the token a to the right of position i, a deletion removes the token at position i, and a substitution replaces the token at position i with a.

$$u_t(\mathrm{ins}(x,i,a) \mid x) = \lambda^{\mathrm{ins}}_{t,i}(x)\, Q^{\mathrm{ins}}_{t,i}(a \mid x), \quad u_t(\mathrm{del}(x,i) \mid x) = \lambda^{\mathrm{del}}_{t,i}(x), \quad u_t(\mathrm{sub}(x,i,a) \mid x) = \lambda^{\mathrm{sub}}_{t,i}(x)\, Q^{\mathrm{sub}}_{t,i}(a \mid x)$$

Each insertion rate is an intensity $\lambda^{\mathrm{ins}}_{t,i}(x)$ times a token distribution $Q^{\mathrm{ins}}_{t,i}(a \mid x)$, each deletion rate is an intensity $\lambda^{\mathrm{del}}_{t,i}(x)$, and each substitution rate is an intensity $\lambda^{\mathrm{sub}}_{t,i}(x)$ times a token distribution $Q^{\mathrm{sub}}_{t,i}(a \mid x)$.

We predict three intensities and two token distributions per position.

In Edit Flows we also allow insertions and deletions

In Edit Flows, there are three edits at each position i:

$$\mathrm{ins}(x, i, a), \qquad \mathrm{del}(x, i), \qquad \mathrm{sub}(x, i, a)$$

An insertion places the token a to the right of position i, a deletion removes the token at position i, and a substitution replaces the token at position i with a.

$$u_t(\mathrm{ins}(x,i,a) \mid x) = \lambda^{\mathrm{ins}}_{t,i}(x)\, Q^{\mathrm{ins}}_{t,i}(a \mid x), \quad u_t(\mathrm{del}(x,i) \mid x) = \lambda^{\mathrm{del}}_{t,i}(x), \quad u_t(\mathrm{sub}(x,i,a) \mid x) = \lambda^{\mathrm{sub}}_{t,i}(x)\, Q^{\mathrm{sub}}_{t,i}(a \mid x)$$

Each insertion rate is an intensity $\lambda^{\mathrm{ins}}_{t,i}(x)$ times a token distribution $Q^{\mathrm{ins}}_{t,i}(a \mid x)$, each deletion rate is an intensity $\lambda^{\mathrm{del}}_{t,i}(x)$, and each substitution rate is an intensity $\lambda^{\mathrm{sub}}_{t,i}(x)$ times a token distribution $Q^{\mathrm{sub}}_{t,i}(a \mid x)$.

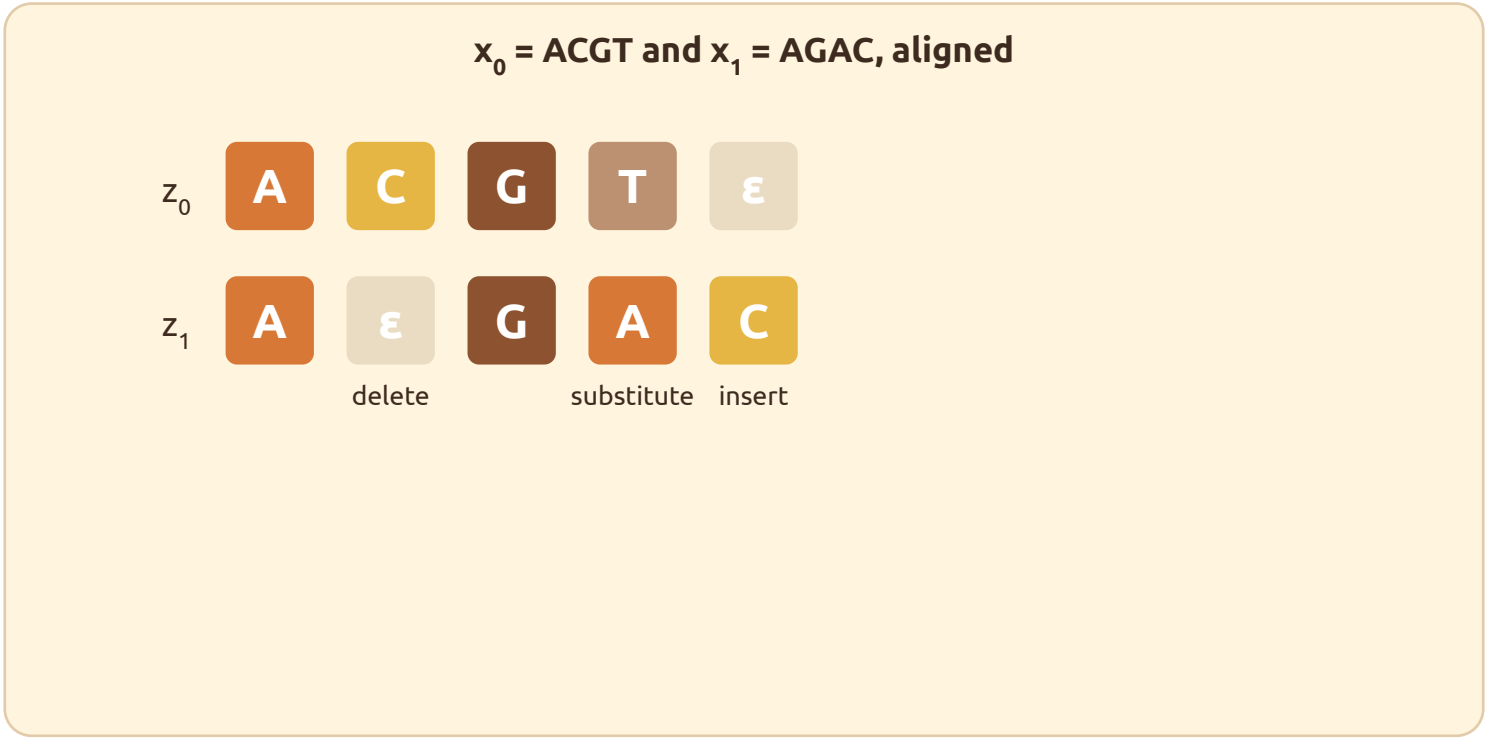
We predict three intensities and two token distributions per position. We set the diagonal to minus the sum of all intensities.

Now align two sequences of different lengths

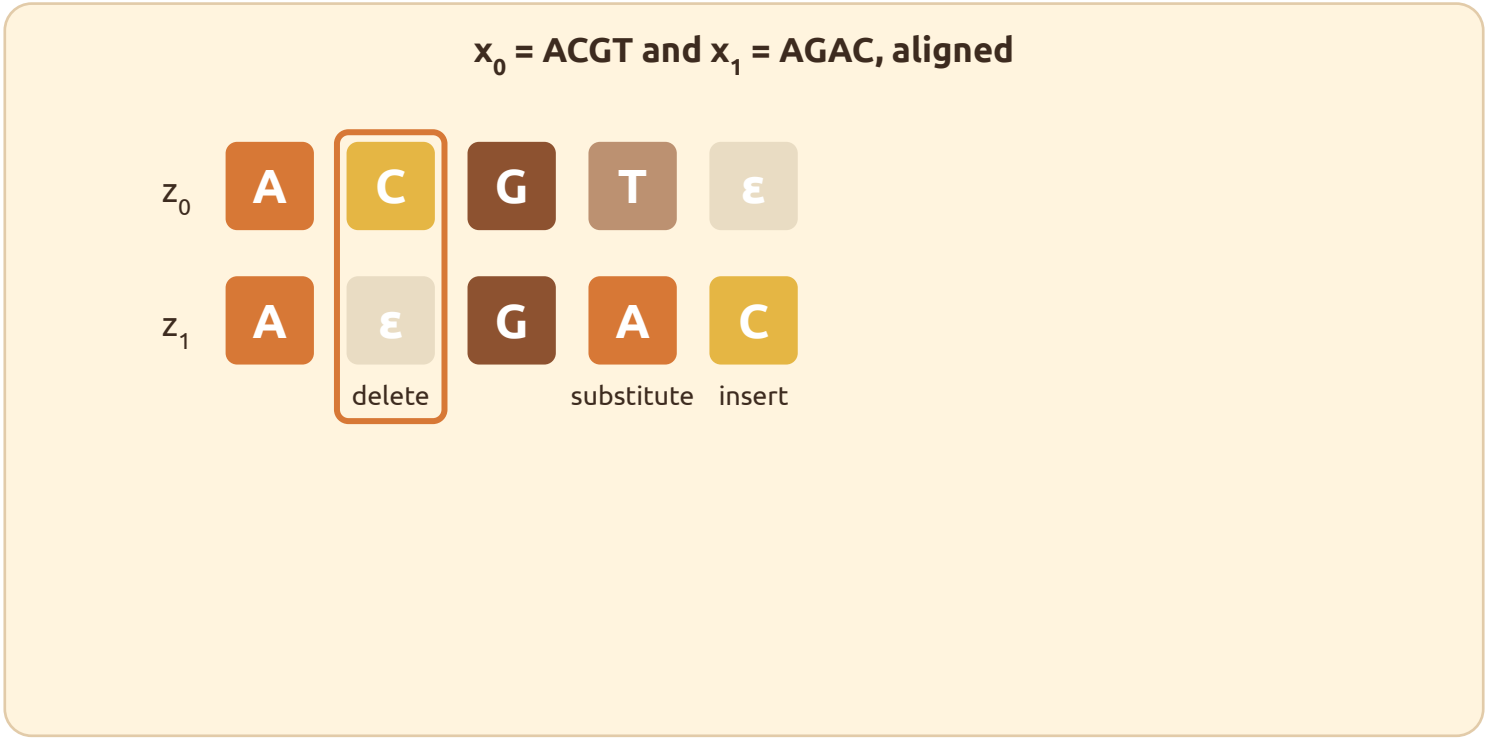
x_0 = ACGT and x_1 = AGAC, aligned

delete substitute insert

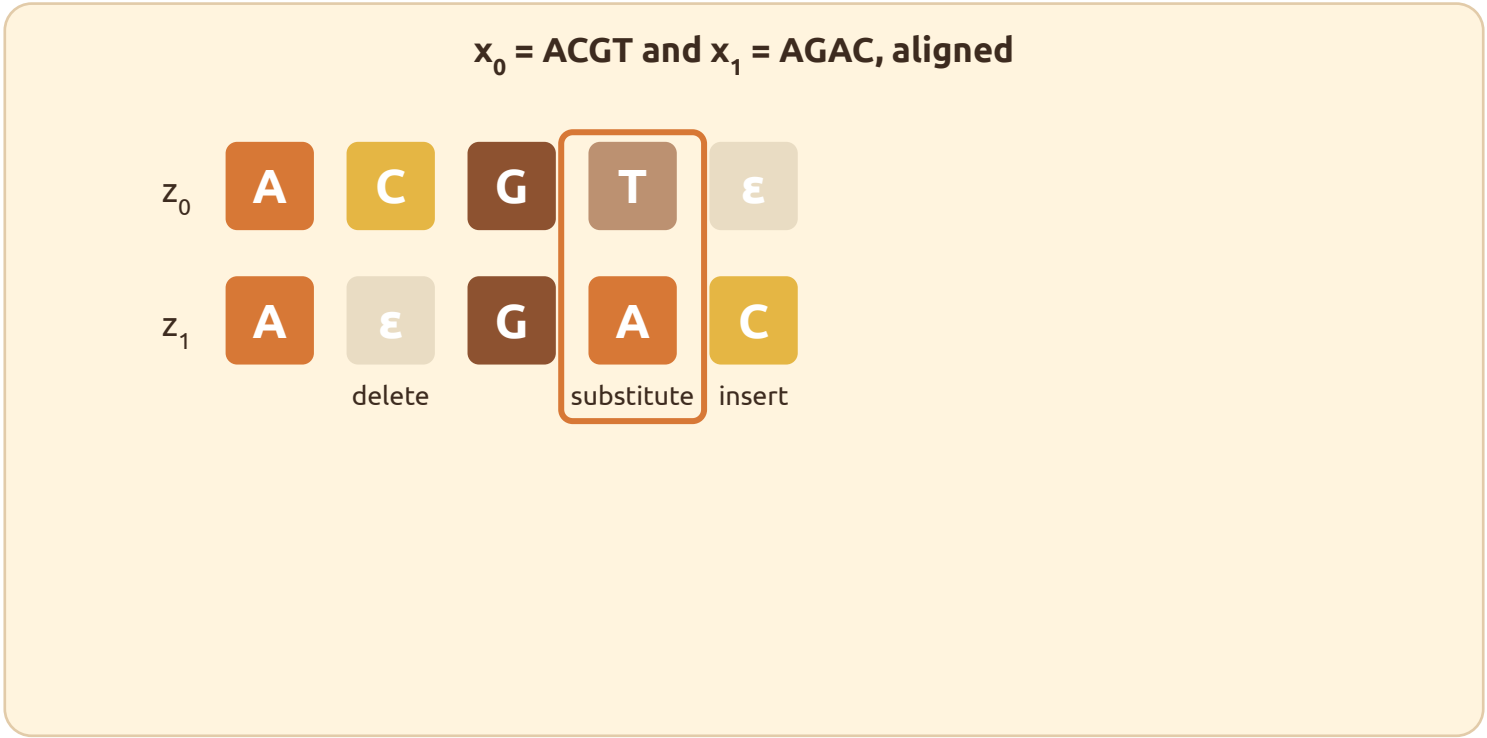
Pad both sequences with a blank ε so that the columns line up



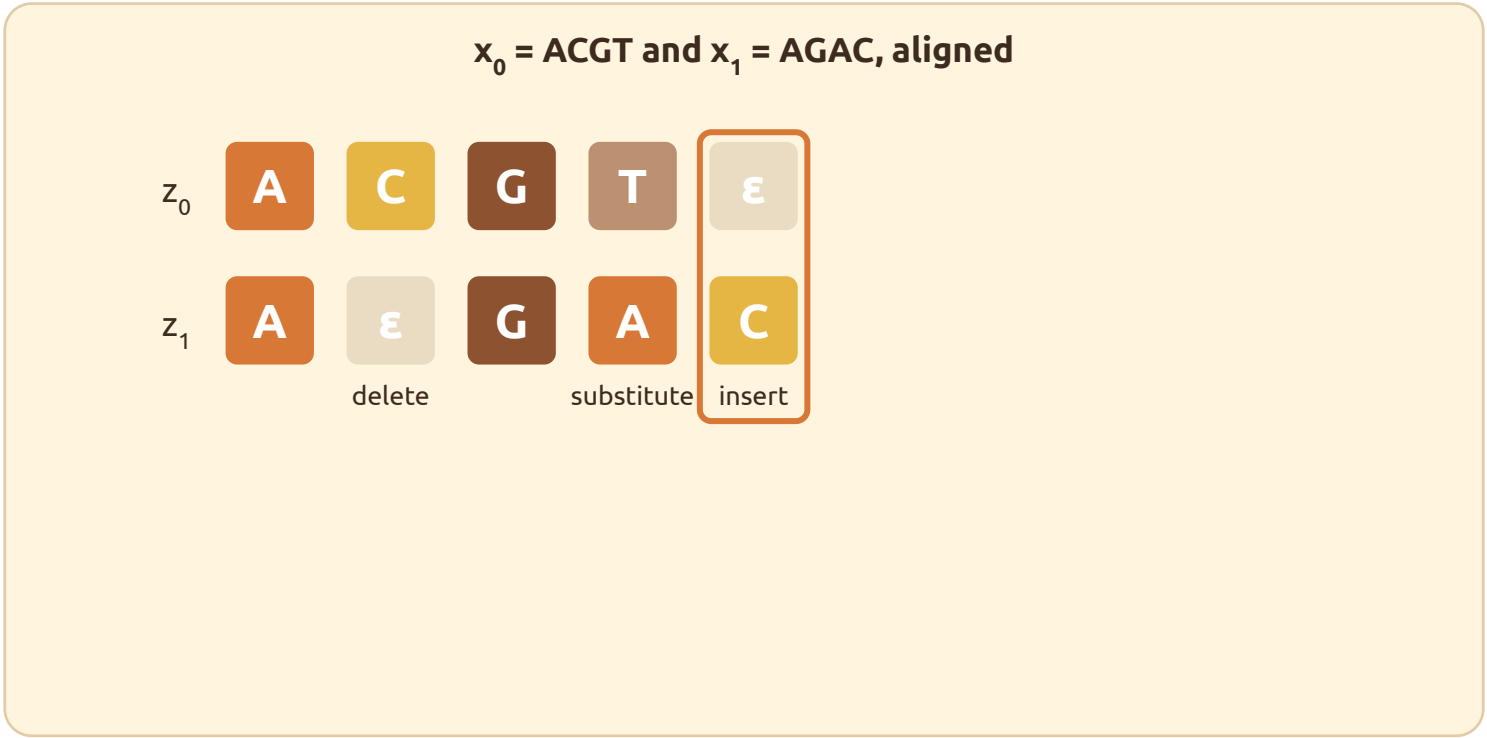
A blank in the target column is a deletion



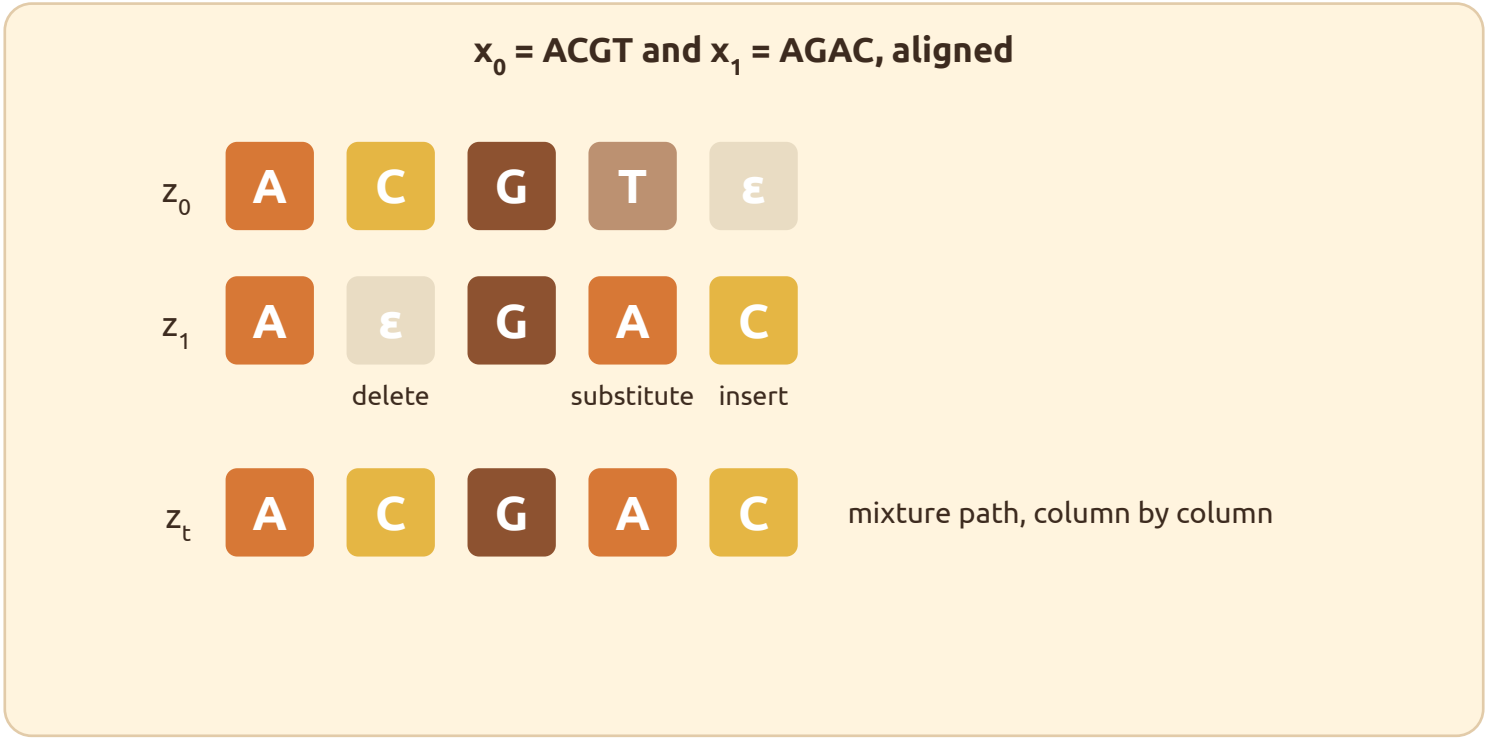
Different tokens in one column are a substitution



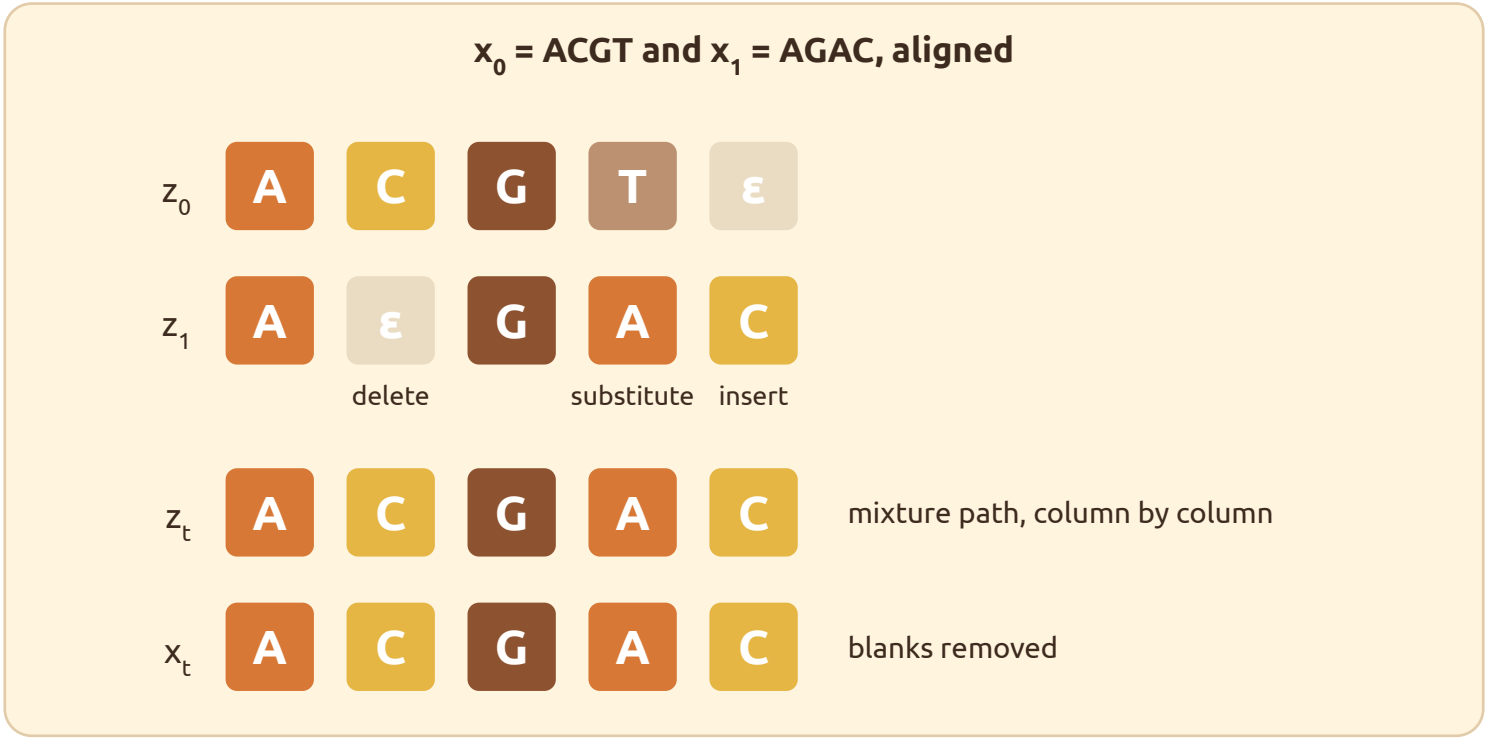
A blank in the source column is an insertion



On the aligned columns, each column follows our mixture path



Remove the blanks from z_t to get the sequence x_t



We average over the alignments we cannot see

We average over the alignments we cannot see

Several alignments can represent the same sequence pair. We cannot determine the current alignment from the noisy sequence x_t alone.

We average over the alignments we cannot see

Several alignments can represent the same sequence pair. We cannot determine the current alignment from the noisy sequence x_t alone.

By their Theorem 3.1, we marginalize the auxiliary alignment, as we marginalized x_1:

$$u_t(x \mid x_t) = \sum_z \mathbb{E}_{p_t(z_t|x_t)} \left[u_t(x, z \mid x_t, z_t) \right]$$

We average over the alignments we cannot see

Several alignments can represent the same sequence pair. We cannot determine the current alignment from the noisy sequence x_t alone.

By their Theorem 3.1, we marginalize the auxiliary alignment, as we marginalized x_1:

$$u_t(x \mid x_t) = \sum_z \mathbb{E}_{p_t(z_t|x_t)} \left[u_t(x, z \mid x_t, z_t) \right]$$

The marginal rate from x_t to x equals the sum over aligned states z of the expected rate on the joint space, taken over the posterior of the current aligned state z_t given the visible sequence x_t.

We average over the alignments we cannot see

Several alignments can represent the same sequence pair. We cannot determine the current alignment from the noisy sequence x_t alone.

By their Theorem 3.1, we marginalize the auxiliary alignment, as we marginalized x_1:

$$u_t(x \mid x_t) = \sum_z \mathbb{E}_{p_t(z_t|x_t)} \left[u_t(x, z \mid x_t, z_t) \right]$$

The marginal rate from x_t to x equals the sum over aligned states z of the expected rate on the joint space, taken over the posterior of the current aligned state z_t given the visible sequence x_t.

It is the posterior-averaging theorem again, with an unseen alignment in place of the unseen data sequence.

We average over the alignments we cannot see

Several alignments can represent the same sequence pair. We cannot determine the current alignment from the noisy sequence x_t alone.

By their Theorem 3.1, we marginalize the auxiliary alignment, as we marginalized x_1:

$$u_t(x \mid x_t) = \sum_z \mathbb{E}_{p_t(z_t|x_t)} \left[u_t(x, z \mid x_t, z_t) \right]$$

The marginal rate from x_t to x equals the sum over aligned states z of the expected rate on the joint space, taken over the posterior of the current aligned state z_t given the visible sequence x_t.

It is the posterior-averaging theorem again, with an unseen alignment in place of the unseen data sequence.

If we train on single sampled alignments, the gradients are the same as for the averaged rate.

We average over the alignments we cannot see

Several alignments can represent the same sequence pair. We cannot determine the current alignment from the noisy sequence x_t alone.

By their Theorem 3.1, we marginalize the auxiliary alignment, as we marginalized x_1:

$$u_t(x \mid x_t) = \sum_z \mathbb{E}_{p_t(z_t|x_t)} \left[u_t(x, z \mid x_t, z_t) \right]$$

The marginal rate from x_t to x equals the sum over aligned states z of the expected rate on the joint space, taken over the posterior of the current aligned state z_t given the visible sequence x_t.

It is the posterior-averaging theorem again, with an unseen alignment in place of the unseen data sequence.

A visible edit can correspond to several alignments. We average their rates using the posterior probabilities of those alignments.

Next, we write the training loss

Next, we write the training loss

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\sum_{x \neq x_t} u_t^\theta(x \mid x_t) \; - \; \sum_{i=1}^{N} \mathbb{1}\big[z_1^i \neq z_t^i\big] \, \frac{\dot{\kappa}_t}{1-\kappa_t} \, \log u_t^\theta\big(x(z_t, i, z_1^i) \mid x_t\big) \Big]$$

Next, we write the training loss

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\sum_{x \neq x_t} u_t^\theta(x \mid x_t) \; - \; \sum_{i=1}^{N} \mathbb{1}\big[z_1^i \neq z_t^i\big] \, \frac{\dot{\kappa}_t}{1-\kappa_t} \, \log u_t^\theta\big(x(z_t, i, z_1^i) \mid x_t\big) \Big]$$

The loss is the expected total rate that the model assigns to leaving x_t, minus the sum over unfinished aligned columns i of $\dot{\kappa}_t$ divided by one minus κ_t, times the log of the model rate of the edit that sets column i to its target.

Next, we write the training loss

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\sum_{x \neq x_t} u_t^\theta(x \mid x_t) \; - \; \sum_{i=1}^{N} \mathbb{1}\big[z_1^i \neq z_t^i\big] \, \frac{\dot{\kappa}_t}{1 - \kappa_t} \, \log u_t^\theta\big(x(z_t, i, z_1^i) \mid x_t\big) \Big]$$

The loss is the expected total rate that the model assigns to leaving x_t, minus the sum over unfinished aligned columns i of $\dot{\kappa}_t$ divided by one minus κ_t, times the log of the model rate of the edit that sets column i to its target.

The first term sums the model's intensities. The second is a weighted cross-entropy over edits that move x_t toward x_1.

Next, we write the training loss

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\sum_{x \neq x_t} u_t^\theta(x \mid x_t) \; - \; \sum_{i=1}^{N} \mathbb{1}\big[z_1^i \neq z_t^i\big] \, \frac{\dot{\kappa}_t}{1 - \kappa_t} \, \log u_t^\theta\big(x(z_t, i, z_1^i) \mid x_t\big) \Big]$$

The loss is the expected total rate that the model assigns to leaving x_t, minus the sum over unfinished aligned columns i of $\dot{\kappa}_t$ divided by one minus κ_t, times the log of the model rate of the edit that sets column i to its target.

The first term sums the model's intensities. The second is a weighted cross-entropy over edits that move x_t toward x_1.

With substitutions only and the mask source, the loss reduces to masked flow matching.

Next, we write the training loss

$$\mathcal{L}(\theta) = \mathbb{E}\Big[\sum_{x \neq x_t} u_t^\theta(x \mid x_t) \; - \; \sum_{i=1}^{N} \mathbb{1}\big[z_1^i \neq z_t^i\big] \, \frac{\dot{\kappa}_t}{1 - \kappa_t} \, \log u_t^\theta\big(x(z_t, i, z_1^i) \mid x_t\big) \Big]$$

The loss is the expected total rate that the model assigns to leaving x_t, minus the sum over unfinished aligned columns i of $\dot{\kappa}_t$ divided by one minus κ_t, times the log of the model rate of the edit that sets column i to its target.

The first term sums the model's intensities. The second is a weighted cross-entropy over edits that move x_t toward x_1.

We train both when to edit and what to change by rewarding the required edits and penalizing excessive editing.

Now take one step of the sampler

Now take one step of the sampler

At each position, insert with probability $h\lambda^{ins}_{t,i}$, and delete or substitute with probability $h(\lambda^{del}_{t,i} + \lambda^{sub}_{t,i})$.

Now take one step of the sampler

At each position, insert with probability $h\lambda^{ins}_{t,i}$, and delete or substitute with probability $h(\lambda^{del}_{t,i} + \lambda^{sub}_{t,i})$.

We cannot both delete and substitute the same token, so we choose between those operations in proportion to their intensities.

Now take one step of the sampler

At each position, insert with probability $h\lambda^{ins}_{t,i}$, and delete or substitute with probability $h(\lambda^{del}_{t,i} + \lambda^{sub}_{t,i})$.

We cannot both delete and substitute the same token, so we choose between those operations in proportion to their intensities.

We draw new tokens from Q^{ins} or Q^{sub}, then apply the selected edits together.

Now take one step of the sampler

At each position, insert with probability $h\lambda^{ins}_{t,i}$, and delete or substitute with probability $h(\lambda^{del}_{t,i} + \lambda^{sub}_{t,i})$.

We cannot both delete and substitute the same token, so we choose between those operations in proportion to their intensities.

We draw new tokens from Q^{ins} or Q^{sub}, then apply the selected edits together.

For x_t = ACG, let $\lambda^{del} = 0.9$ and $\lambda^{sub} = 0.3$ at C. A step of $h = 0.05$ edits C with probability 0.06. Conditional on an edit, deletion has probability 0.75.

Edit Flows: what happens to ACG?

At the C in a toy ACG sequence, deletion has rate 0.9 and substitution has rate 0.3. Take a first-order step of 0.05, with no insertions. A substitution chooses T with probability 0.8 and A otherwise.

Edit Flows: what happens to ACG?

At the C in a toy ACG sequence, deletion has rate 0.9 and substitution has rate 0.3. Take a first-order step of 0.05, with no insertions. A substitution chooses T with probability 0.8 and A otherwise.

$$p(\text{delete}) = 0.05(0.9) = 0.045, \quad p(\text{substitute}) = 0.05(0.3) = 0.015, \quad p(\text{keep}) = 0.94$$

Edit Flows: what happens to ACG?

At the C in a toy ACG sequence, deletion has rate 0.9 and substitution has rate 0.3. Take a first-order step of 0.05, with no insertions. A substitution chooses T with probability 0.8 and A otherwise.

$$p(\text{delete}) = 0.05(0.9) = 0.045, \quad p(\text{substitute}) = 0.05(0.3) = 0.015, \quad p(\text{keep}) = 0.94$$

Deletion produces AG. A substitution edits the C without changing the sequence length.

Edit Flows: what happens to ACG?

At the C in a toy ACG sequence, deletion has rate 0.9 and substitution has rate 0.3. Take a first-order step of 0.05, with no insertions. A substitution chooses T with probability 0.8 and A otherwise.

$$p(\text{delete}) = 0.05(0.9) = 0.045, \quad p(\text{substitute}) = 0.05(0.3) = 0.015, \quad p(\text{keep}) = 0.94$$

Deletion produces AG. A substitution edits the C without changing the sequence length.

$$p(\text{ATG}) = p(\text{substitute})\, Q^{\text{sub}}(\text{T}) = (0.015)(0.8) = 0.012$$

Edit Flows: what happens to ACG?

At the C in a toy ACG sequence, deletion has rate 0.9 and substitution has rate 0.3. Take a first-order step of 0.05, with no insertions. A substitution chooses T with probability 0.8 and A otherwise.

$$p(\text{delete}) = 0.05(0.9) = 0.045, \quad p(\text{substitute}) = 0.05(0.3) = 0.015, \quad p(\text{keep}) = 0.94$$

Deletion produces AG. A substitution edits the C without changing the sequence length.

$$p(\text{ATG}) = p(\text{substitute})\, Q^{\text{sub}}(\text{T}) = (0.015)(0.8) = 0.012$$

The probability of ATG is 1.2%, while the probability of AG is 4.5%. ACG remains unchanged with probability 94%.

Edit Flows: what happens to ACG?

At the C in a toy ACG sequence, deletion has rate 0.9 and substitution has rate 0.3. Take a first-order step of 0.05, with no insertions. A substitution chooses T with probability 0.8 and A otherwise.

$$p(\text{delete}) = 0.05(0.9) = 0.045, \quad p(\text{substitute}) = 0.05(0.3) = 0.015, \quad p(\text{keep}) = 0.94$$

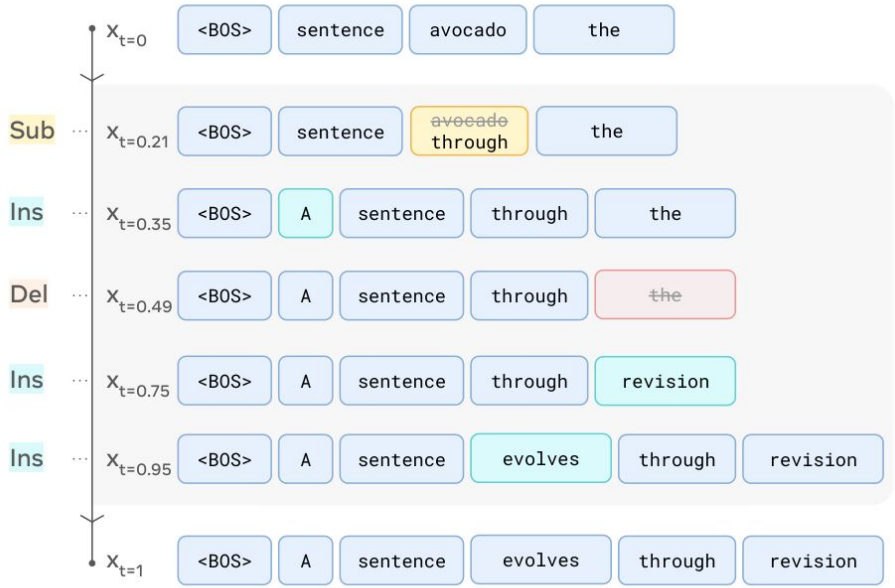
Deletion produces AG. A substitution edits the C without changing the sequence length.

$$p(\text{ATG}) = p(\text{substitute})\, Q^{\text{sub}}(\text{T}) = (0.015)(0.8) = 0.012$$

The probability of ATG is 1.2%, while the probability of AG is 4.5%. ACG remains unchanged with probability 94%.

We use the intensity to determine whether to edit. If the edit needs a new token, we draw it from the token distribution.

Each step is one insertion, deletion, or substitution



Generation from a short random sequence, with each edit labeled.

Havasi,..,Chen. *NeurIPS* (2025)

Now compare Edit Flows with a mask model on code

	Method	HumanEval		HumanEval+		MBPP	
		Pass@1	Pass@10	Pass@1	Pass@10	Pass@1	Pass@10
	Autoregressive (Gat et al., 2024)	14.3	21.3			17.0	34.3
	Llama3 Autoregressive†	17.0	34.7	14.0	28.6	25.6	45.4
Non-AR	Mask DFM (Gat et al., 2024)	6.7	13.4			6.7	20.6
Non-AR	Mask DFM (Oracle Length) (Gat et al., 2024)	11.6	18.3			13.1	28.4
Non-AR	Mask DFM†	9.1	17.6	7.9	13.4	6.2	25.0
Non-AR	Uniform X_0 + Edit Flow (**Ours**)	9.7	24.3	9.7	19.5	9.4	33.4
Non-AR	Edit Flow (**Ours**)	12.8	24.3	10.4	20.7	10.0	36.4
Non-AR	Localized Edit Flow (**Ours**)	14.0	22.6	10.4	18.9	14.8	34.0

Table 3: Code generation benchmarks using Llama3 1.3B parameter models trained on the CodeLlama (Roziere et al., 2023) datamix. †Superscript denotes our own implementation. We highlight the best non-autoregressive models, where colors show the best and second best among each metric.

Havasi,..,Chen. *NeurIPS* (2025)

A mask model of the same size solves 9.1% of HumanEval problems on the first try

	Method	HumanEval		HumanEval+		MBPP	
		Pass@1	Pass@10	Pass@1	Pass@10	Pass@1	Pass@10
	Autoregressive (Gat et al., 2024)	14.3	21.3			17.0	34.3
	Llama3 Autoregressive†	17.0	34.7	14.0	28.6	25.6	45.4
Non-AR	Mask DFM (Gat et al., 2024)	6.7	13.4			6.7	20.6
Non-AR	Mask DFM (Oracle Length) (Gat et al., 2024)	11.6	18.3			13.1	28.4
Non-AR	Mask DFM†	9.1	17.6	7.9	13.4	6.2	25.0
Non-AR	Uniform X_0 + Edit Flow (**Ours**)	9.7	24.3	9.7	19.5	9.4	33.4
Non-AR	Edit Flow (**Ours**)	12.8	24.3	10.4	20.7	10.0	36.4
Non-AR	Localized Edit Flow (**Ours**)	14.0	22.6	10.4	18.9	14.8	34.0

Table 3: Code generation benchmarks using Llama3 1.3B parameter models trained on the CodeLlama (Roziere et al., 2023) datamix. †Superscript denotes our own implementation. We highlight the best non-autoregressive models, where colors show the best and second best among each metric.

Havasi,..,Chen. *NeurIPS* (2025)

Edit Flows solve 12.8%, more than the mask model given the true length

	Method	HumanEval		HumanEval+		MBPP	
		Pass@1	Pass@10	Pass@1	Pass@10	Pass@1	Pass@10
	Autoregressive (Gat et al., 2024)	14.3	21.3			17.0	34.3
	Llama3 Autoregressive†	17.0	34.7	14.0	28.6	25.6	45.4
Non-AR	Mask DFM (Gat et al., 2024)	6.7	13.4			6.7	20.6
Non-AR	Mask DFM (Oracle Length) (Gat et al., 2024)	11.6	18.3			13.1	28.4
Non-AR	Mask DFM†	9.1	17.6	7.9	13.4	6.2	25.0
Non-AR	Uniform X_0 + Edit Flow (**Ours**)	9.7	24.3	9.7	19.5	9.4	33.4
Non-AR	Edit Flow (**Ours**)	12.8	24.3	10.4	20.7	10.0	36.4
Non-AR	Localized Edit Flow (**Ours**)	14.0	22.6	10.4	18.9	14.8	34.0

Table 3: Code generation benchmarks using Llama3 1.3B parameter models trained on the CodeLlama (Roziere et al., 2023) datamix. †Superscript denotes our own implementation. We highlight the best non-autoregressive models, where colors show the best and second best among each metric.

Havasi,..,Chen. *NeurIPS* (2025)

Letting insertions favor neighbors of existing tokens raises MBPP from 10.0% to 14.8%

	Method	HumanEval		HumanEval+		MBPP	
		Pass@1	Pass@10	Pass@1	Pass@10	Pass@1	Pass@10
	Autoregressive (Gat et al., 2024)	14.3	21.3			17.0	34.3
	Llama3 Autoregressive†	17.0	34.7	14.0	28.6	25.6	45.4
Non-AR	Mask DFM (Gat et al., 2024)	6.7	13.4			6.7	20.6
Non-AR	Mask DFM (Oracle Length) (Gat et al., 2024)	11.6	18.3			13.1	28.4
Non-AR	Mask DFM†	9.1	17.6	7.9	13.4	6.2	25.0
Non-AR	Uniform X_0 + Edit Flow (**Ours**)	9.7	24.3	9.7	19.5	9.4	33.4
Non-AR	Edit Flow (**Ours**)	12.8	24.3	10.4	20.7	10.0	36.4
Non-AR	Localized Edit Flow (**Ours**)	14.0	22.6	10.4	18.9	14.8	34.0

Table 3: Code generation benchmarks using Llama3 1.3B parameter models trained on the CodeLlama (Roziere et al., 2023) datamix. †Superscript denotes our own implementation. We highlight the best non-autoregressive models, where colors show the best and second best among each metric.

Havasi,..,Chen. *NeurIPS* (2025)

We can use the same rate-based construction on sequences of varying length

	Method	HumanEval		HumanEval+		MBPP	
		Pass@1	Pass@10	Pass@1	Pass@10	Pass@1	Pass@10
	Autoregressive (Gat et al., 2024)	14.3	21.3			17.0	34.3
	Llama3 Autoregressive†	17.0	34.7	14.0	28.6	25.6	45.4
Non-AR	Mask DFM (Gat et al., 2024)	6.7	13.4			6.7	20.6
Non-AR	Mask DFM (Oracle Length) (Gat et al., 2024)	11.6	18.3			13.1	28.4
Non-AR	Mask DFM†	9.1	17.6	7.9	13.4	6.2	25.0
Non-AR	Uniform X_0 + Edit Flow (**Ours**)	9.7	24.3	9.7	19.5	9.4	33.4
Non-AR	Edit Flow (**Ours**)	12.8	24.3	10.4	20.7	10.0	36.4
Non-AR	Localized Edit Flow (**Ours**)	14.0	22.6	10.4	18.9	14.8	34.0

Table 3: Code generation benchmarks using Llama3 1.3B parameter models trained on the CodeLlama (Roziere et al., 2023) datamix. †Superscript denotes our own implementation. We highlight the best non-autoregressive models, where colors show the best and second best among each metric.

Havasi,..,Chen. *NeurIPS* (2025)

Summary: velocities, paths, and the denoiser

Summary: velocities, paths, and the denoiser

Velocities are rates: $u_t(y, x) \geq 0 \ (y \neq x), \quad \sum_y u_t(y, x) = 0$

Summary: velocities, paths, and the denoiser

Velocities are rates: $u_t(y, x) \geq 0 \ (y \neq x), \quad \sum_y u_t(y, x) = 0$

A probability velocity has nonnegative rates toward other states. Including the diagonal, its rates from each state sum to zero.

Summary: velocities, paths, and the denoiser

Velocities are rates: $$u_t(y, x) \geq 0 \ (y \neq x), \quad \sum_y u_t(y, x) = 0$$

They generate a path: $$\frac{d}{dt} p_t(y) = \sum_x u_t(y, x)\, p_t(x)$$

Summary: velocities, paths, and the denoiser

Velocities are rates: $$u_t(y, x) \geq 0 \ (y \neq x), \quad \sum_y u_t(y, x) = 0$$

They generate a path: $$\frac{d}{dt} p_t(y) = \sum_x u_t(y, x)\, p_t(x)$$

A velocity generates p_t when the rate of change of $p_t(y)$ equals the sum over x of $u_t(y, x)$ times $p_t(x)$.

Summary: velocities, paths, and the denoiser

Velocities are rates: $$u_t(y, x) \geq 0 \ (y \neq x), \quad \sum_y u_t(y, x) = 0$$

They generate a path: $$\frac{d}{dt} p_t(y) = \sum_x u_t(y, x)\, p_t(x)$$

Mixture paths: $$p_t(x^i \mid x_0, x_1) = (1 - \kappa_t)\, \delta_{x_0^i} + \kappa_t\, \delta_{x_1^i}$$

Summary: velocities, paths, and the denoiser

Velocities are rates: $$u_t(y, x) \geq 0 \ (y \neq x), \quad \sum_y u_t(y, x) = 0$$

They generate a path: $$\frac{d}{dt} p_t(y) = \sum_x u_t(y, x)\, p_t(x)$$

Mixture paths: $$p_t(x^i \mid x_0, x_1) = (1 - \kappa_t)\, \delta_{x_0^i} + \kappa_t\, \delta_{x_1^i}$$

Each position shows its source token with probability one minus κ_t and its data token with probability κ_t.

Summary: training, sampling, and correctors

Summary: training, sampling, and correctors

Posterior average: $$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i} \right]$$

Summary: training, sampling, and correctors

Posterior average: $$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i} \right]$$

The marginal velocity is the schedule rate times the denoiser distribution minus the point mass on the current token.

Summary: training, sampling, and correctors

Posterior average:

$$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i} \right]$$

Training:

$$\mathcal{L} = -\,\mathbb{E} \sum_i \log p^{\theta}_{1|t}(X_1^i \mid X_t)$$

Summary: training, sampling, and correctors

Posterior average:

$$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i} \right]$$

Training:

$$\mathcal{L} = -\,\mathbb{E} \sum_i \log p^{\theta}_{1|t}(X_1^i \mid X_t)$$

We train the denoiser by cross-entropy on the true data token at every position.

Summary: training, sampling, and correctors

Posterior average:

$$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i} \right]$$

Training:

$$\mathcal{L} = -\,\mathbb{E} \sum_i \log p^\theta_{1|t}(X_1^i \mid X_t)$$

Correctors:

$$\bar{u}_t = \hat{u}_t + \beta_t \left(\hat{u}_t - \breve{u}_t \right)$$

Summary: training, sampling, and correctors

Posterior average: $$u_t^i(\cdot, x) = \frac{\dot{\kappa}_t}{1 - \kappa_t} \left[p_{1|t}(\cdot \mid x) - \delta_{x^i} \right]$$

Training: $$\mathcal{L} = -\,\mathbb{E} \sum_i \log p^\theta_{1|t}(X_1^i \mid X_t)$$

Correctors: $$\bar{u}_t = \hat{u}_t + \beta_t \left(\hat{u}_t - \check{u}_t \right)$$

The corrector velocity $\bar{u}_t$ equals the forward velocity $\hat{u}_t$ plus β_t times the difference between the forward velocity $\hat{u}_t$ and the backward velocity $\check{u}_t$.

Continuous representations of discrete tokens

Continuous representations of discrete tokens

So far, each position has held one of four bases. We described its changes through jump rates.

Continuous representations of discrete tokens

So far, each position has held one of four bases. We described its changes through jump rates.

The master equation, the posterior average, and the corrector all work on that discrete set.

Continuous representations of discrete tokens

So far, each position has held one of four bases. We described its changes through jump rates.

The master equation, the posterior average, and the corrector all work on that discrete set.

With continuous flow matching, we can use ODE solvers, classifier guidance, and distillation to one step.

Now we relax a token into a point we can move continuously

Now we relax a token into a point we can move continuously

A base becomes a probability vector over A, C, G, and T.

Now we relax a token into a point we can move continuously

A base becomes a probability vector over A, C, G, and T.

Certainty about A corresponds to the corner (1, 0, 0, 0), while complete uncertainty corresponds to the center (0.25, 0.25, 0.25, 0.25).

Now we relax a token into a point we can move continuously

A base becomes a probability vector over A, C, G, and T.

Certainty about A corresponds to the corner (1, 0, 0, 0), while complete uncertainty corresponds to the center (0.25, 0.25, 0.25, 0.25).

Generation becomes a continuous curve from the center to a corner, driven by an ordinary velocity field.

Now we relax a token into a point we can move continuously

A base becomes a probability vector over A, C, G, and T.

Certainty about A corresponds to the corner (1, 0, 0, 0), while complete uncertainty corresponds to the center (0.25, 0.25, 0.25, 0.25).

Generation becomes a continuous curve from the center to a corner, driven by an ordinary velocity field.

At the end we read off the token by taking the largest coordinate.

The simplex is the set of probability vectors

The simplex is the set of probability vectors

With K letters in the alphabet, $$S_K = \{ x \in \mathbb{R}^K \ : \ \sum_k x_k = 1, \ x_k \geq 0 \}$$

The simplex is the set of probability vectors

With K letters in the alphabet, $$S_K = \{ x \in \mathbb{R}^K \ : \ \sum_k x_k = 1, \ x_k \geq 0 \}$$

The simplex S_K is the set of vectors x in K-dimensional space whose coordinates sum to one and are all nonnegative.

The simplex is the set of probability vectors

With K letters in the alphabet, $$S_K = \{ x \in \mathbb{R}^K \ : \ \sum_k x_k = 1, \ x_k \geq 0 \}$$

The simplex S_K is the set of vectors x in K-dimensional space whose coordinates sum to one and are all nonnegative.

For DNA, K = 4 and S_4 is a solid tetrahedron with corners A, C, G, and T.

The simplex is the set of probability vectors

With K letters in the alphabet, $$S_K = \{ x \in \mathbb{R}^K \ : \ \sum_k x_k = 1, \ x_k \geq 0 \}$$

The simplex S_K is the set of vectors x in K-dimensional space whose coordinates sum to one and are all nonnegative.

For DNA, K = 4 and S_4 is a solid tetrahedron with corners A, C, G, and T.

The corners are the one-hot vectors: $e_k \in S_K, \qquad (e_k)_j = \delta_{jk}$

The simplex is the set of probability vectors

With K letters in the alphabet, $$S_K = \{\, x \in \mathbb{R}^K \;:\; \textstyle\sum_k x_k = 1,\; x_k \geq 0 \,\}$$

The simplex S_K is the set of vectors x in K-dimensional space whose coordinates sum to one and are all nonnegative.

For DNA, K = 4 and S_4 is a solid tetrahedron with corners A, C, G, and T.

The corners are the one-hot vectors: $$e_k \in S_K, \qquad (e_k)_j = \delta_{jk}$$

The corner e_k is the vector whose k-th coordinate is one and whose other coordinates are zero.

The simplex is the set of probability vectors

With K letters in the alphabet, $$S_K = \{\, x \in \mathbb{R}^K \;:\; \textstyle\sum_k x_k = 1,\; x_k \geq 0 \,\}$$

The simplex S_K is the set of vectors x in K-dimensional space whose coordinates sum to one and are all nonnegative.

For DNA, K = 4 and S_4 is a solid tetrahedron with corners A, C, G, and T.

The corners are the one-hot vectors: $e_k \in S_K, \qquad (e_k)_j = \delta_{jk}$

The sum constraint removes one degree of freedom, so S_K is a flat piece of a K – 1 dimensional plane.

Data tokens occupy the simplex corners

Data tokens occupy the simplex corners

Read a categorical distribution over letters on the simplex: $$q(x) = \sum_{k=1}^{K} q_k \, \delta(x - e_k)$$

Data tokens occupy the simplex corners

Read a categorical distribution over letters on the simplex: $$q(x) = \sum_{k=1}^{K} q_k \, \delta(x - e_k)$$

The data distribution q(x) is the sum over letters k of the probability q_k times a point mass at the corner e_k.

Data tokens occupy the simplex corners

Read a categorical distribution over letters on the simplex: $$q(x) = \sum_{k=1}^{K} q_k \, \delta(x - e_k)$$

The data distribution q(x) is the sum over letters k of the probability q_k times a point mass at the corner e_k.

We keep data tokens at the corners, so we lose no information in the relaxation.

Data tokens occupy the simplex corners

Read a categorical distribution over letters on the simplex: $$q(x) = \sum_{k=1}^{K} q_k \, \delta(x - e_k)$$

The data distribution q(x) is the sum over letters k of the probability q_k times a point mass at the corner e_k.

We keep data tokens at the corners, so we lose no information in the relaxation.

A sequence of N positions is a point in the product S_K^N, one simplex per position.

A path now has to run from the center to a corner

Read a categorical distribution over letters on the simplex: $$q(x) = \sum_{k=1}^{K} q_k \, \delta(x - e_k)$$

The data distribution q(x) is the sum over letters k of the probability q_k times a point mass at the corner e_k.

We keep data tokens at the corners, so we lose no information in the relaxation.

A sequence of N positions is a point in the product S_K^N, one simplex per position.

A straight-line path on the simplex

A straight-line path on the simplex

Interpolate a source point and a corner: $\psi_t(x_0 \mid x_1) = (1 - t)\, x_0 + t\, x_1$

A straight-line path on the simplex

Interpolate a source point and a corner: $$\psi_t(x_0 \mid x_1) = (1 - t)\, x_0 + t\, x_1$$

The straight-line interpolant ψ_t equals one minus t times the source point x_0, plus t times the data corner x_1.

A straight-line path on the simplex

Interpolate a source point and a corner: $\psi_t(x_0 \mid x_1) = (1 - t)\, x_0 + t\, x_1$

The straight-line interpolant ψ_t equals one minus t times the source point x_0, plus t times the data corner x_1.

Its velocity is the one we know: $u_t(x \mid x_1) = \dfrac{x_1 - x}{1 - t}$

A straight-line path on the simplex

Interpolate a source point and a corner: $\psi_t(x_0 \mid x_1) = (1 - t)\, x_0 + t\, x_1$

The straight-line interpolant ψ_t equals one minus t times the source point x_0, plus t times the data corner x_1.

Its velocity is the one we know: $u_t(x \mid x_1) = \dfrac{x_1 - x}{1 - t}$

The conditional velocity at the point x equals the displacement x_1 – x that remains to the corner, divided by the time 1 – t that remains.

A straight-line path on the simplex

Interpolate a source point and a corner: $\psi_t(x_0 \mid x_1) = (1 - t)\, x_0 + t\, x_1$

The straight-line interpolant ψ_t equals one minus t times the source point x_0, plus t times the data corner x_1.

Its velocity is the one we know: $u_t(x \mid x_1) = \dfrac{x_1 - x}{1 - t}$

The conditional velocity at the point x equals the displacement x_1 – x that remains to the corner, divided by the time 1 – t that remains.

The entire path stays inside the simplex because every point is a convex combination of two simplex points.

Draw the source point uniformly on the simplex

Draw the source point uniformly on the simplex

Draw x_0 uniformly on the simplex. The interpolant shrinks the whole simplex toward the corner e_i by the factor $1 - t$.

Draw the source point uniformly on the simplex

Draw x_0 uniformly on the simplex. The interpolant shrinks the whole simplex toward the corner e_i by the factor 1 – t.

$$p_t(x \mid x_1 = e_i) = \begin{cases} \left[(1-t)^{K-1}\,\Gamma(K)\right]^{-1} & x_i \geq t \\ 0 & x_i < t \end{cases}$$

Draw the source point uniformly on the simplex

Draw x_0 uniformly on the simplex. The interpolant shrinks the whole simplex toward the corner e_i by the factor 1 – t.

$$p_t(x \mid x_1 = e_i) = \begin{cases} \left[(1-t)^{K-1}\,\Gamma(K)\right]^{-1} & x_i \geq t \\ 0 & x_i < t \end{cases}$$

The conditional density is constant on the region where the coordinate x_i is at least t, with value one over the quantity one minus t raised to the K – 1, times Γ(K), and it is zero where x_i is below t.

Draw the source point uniformly on the simplex

Draw x_0 uniformly on the simplex. The interpolant shrinks the whole simplex toward the corner e_i by the factor 1 – t.

$$p_t(x \mid x_1 = e_i) = \begin{cases} \left[(1-t)^{K-1}\,\Gamma(K)\right]^{-1} & x_i \geq t \\ 0 & x_i < t \end{cases}$$

The conditional density is constant on the region where the coordinate x_i is at least t, with value one over the quantity one minus t raised to the K – 1, times Γ(K), and it is zero where x_i is below t.

The support is the smaller simplex $\{x_i \geq t\}$. We normalize its constant density to keep the total probability equal to one.

Draw the source point uniformly on the simplex

Draw x_0 uniformly on the simplex. The interpolant shrinks the whole simplex toward the corner e_i by the factor 1 – t.

$$p_t(x \mid x_1 = e_i) = \begin{cases} \left[(1-t)^{K-1}\,\Gamma(K)\right]^{-1} & x_i \geq t \\ 0 & x_i < t \end{cases}$$

The conditional density is constant on the region where the coordinate x_i is at least t, with value one over the quantity one minus t raised to the K – 1, times Γ(K), and it is zero where x_i is below t.

The support is the smaller simplex $\{x_i \geq t\}$. We normalize its constant density to keep the total probability equal to one.

A point x with $x_i < t$ could not have come from corner e_i, so the posterior over corners gives e_i probability zero.

Could a smoother probability path keep alternative tokens available for longer?

Dirichlet Flow Matching

Dirichlet Flow Matching with Applications to DNA Sequence Design

Hannes Stark [*1] **Bowen Jing** [*1] **Chenyu Wang** [1] **Gabriele Corso** [1]
Bonnie Berger [1 2] **Regina Barzilay** [1] **Tommi Jaakkola** [1]

Stark,..,Jaakkola. *ICML* (2024)

How many data corners remain possible?

How many data corners remain possible?

Proposition 1: with the straight-line path, the posterior over corners has support on at most $k - 1$ corners once $t > 1/k$.

How many data corners remain possible?

Proposition 1: with the straight-line path, the posterior over corners has support on at most k – 1 corners once t > 1/k.

Suppose k corners are still possible at time t > 1/k:

$$\mathbf{1}^\top x = \sum_{j=1}^{K} x_j \;\geq\; \sum_{j=1}^{k} x_j \;\geq\; k\,t \;>\; k \cdot \frac{1}{k} = 1$$

How many data corners remain possible?

Proposition 1: with the straight-line path, the posterior over corners has support on at most k – 1 corners once t > 1/k.

Suppose k corners are still possible at time t > 1/k:

$$\mathbf{1}^\top x = \sum_{j=1}^{K} x_j \;\geq\; \sum_{j=1}^{k} x_j \;\geq\; k\,t \;>\; k \cdot \frac{1}{k} = 1$$

The sum of all coordinates of x is at least the sum of the k coordinates that are still possible, each of which is at least t, so the total exceeds k times one over k, which is one.

How many data corners remain possible?

Proposition 1: with the straight-line path, the posterior over corners has support on at most k – 1 corners once t > 1/k.

Suppose k corners are still possible at time t > 1/k:

$$\mathbf{1}^\top x = \sum_{j=1}^{K} x_j \;\geq\; \sum_{j=1}^{k} x_j \;\geq\; k\,t \;>\; k \cdot \frac{1}{k} = 1$$

The sum of all coordinates of x is at least the sum of the k coordinates that are still possible, each of which is at least t, so the total exceeds k times one over k, which is one.

A simplex point sums to exactly one, so k corners cannot all stay possible. The assumption fails.

Let’s put numbers on that for DNA

Let’s put numbers on that for DNA

For DNA, one base is eliminated at t = 1/4, another at t = 1/3, and a third at t = 1/2.

Let’s put numbers on that for DNA

For DNA, one base is eliminated at t = 1/4, another at t = 1/3, and a third at t = 1/2.

At t = 0.3 with x = (0.34, 0.33, 0.33, 0.00), only A, C, and G remain possible, which is the bound for k = 4.

Let’s put numbers on that for DNA

For DNA, one base is eliminated at t = 1/4, another at t = 1/3, and a third at t = 1/2.

At t = 0.3 with x = (0.34, 0.33, 0.33, 0.00), only A, C, and G remain possible, which is the bound for k = 4.

At t = 0.6 with x = (0.70, 0.30, 0, 0), only A remains, so the posterior is the argmax.

Let’s put numbers on that for DNA

For DNA, one base is eliminated at t = 1/4, another at t = 1/3, and a third at t = 1/2.

At t = 0.3 with x = (0.34, 0.33, 0.33, 0.00), only A, C, and G remain possible, which is the bound for k = 4.

At t = 0.6 with x = (0.70, 0.30, 0, 0), only A remains, so the posterior is the argmax.

The target distribution is discontinuous in both time and space. Numerical integration is therefore sensitive to the step size.

Recall the Dirichlet distribution

Recall the Dirichlet distribution

Take a density on the simplex with one parameter per letter:

$$\mathrm{Dir}(x; \alpha_1, \ldots, \alpha_K) = \frac{1}{\mathcal{B}(\alpha)} \prod_{k=1}^{K} x_k^{\alpha_k - 1}$$

Recall the Dirichlet distribution

Take a density on the simplex with one parameter per letter:

$$\mathrm{Dir}(x; \alpha_1, \ldots, \alpha_K) = \frac{1}{\mathcal{B}(\alpha)} \prod_{k=1}^{K} x_k^{\alpha_k - 1}$$

The Dirichlet density at the point x is one over the multivariate beta function of α, times the product over letters of x_k raised to the power $\alpha_k - 1$.

Recall the Dirichlet distribution

Take a density on the simplex with one parameter per letter:

$$\mathrm{Dir}(x; \alpha_1, \ldots, \alpha_K) = \frac{1}{\mathcal{B}(\alpha)} \prod_{k=1}^{K} x_k^{\alpha_k - 1}$$

The Dirichlet density at the point x is one over the multivariate beta function of α, times the product over letters of x_k raised to the power $\alpha_k - 1$.

Setting every concentration to one gives the uniform distribution on the simplex.

Recall the Dirichlet distribution

Take a density on the simplex with one parameter per letter:

$$\mathrm{Dir}(x; \alpha_1, \ldots, \alpha_K) = \frac{1}{\mathcal{B}(\alpha)} \prod_{k=1}^{K} x_k^{\alpha_k - 1}$$

The Dirichlet density at the point x is one over the multivariate beta function of α, times the product over letters of x_k raised to the power $\alpha_k - 1$.

Setting every concentration to one gives the uniform distribution on the simplex.

Increasing one concentration places more probability near the corresponding corner.

Now replace the straight line with a Dirichlet family

Now replace the straight line with a Dirichlet family

Replace the straight line with a family of Dirichlet distributions:

$$\boxed{p_t(x \mid x_1 = e_i) = \mathrm{Dir}\big(x;\ \alpha = \mathbf{1} + t\, e_i\big)}$$

Now replace the straight line with a Dirichlet family

Replace the straight line with a family of Dirichlet distributions:

$$\boxed{p_t(x \mid x_1 = e_i) = \mathrm{Dir}\big(x;\ \alpha = \mathbf{1} + t\, e_i\big)}$$

The conditional path at time t is the Dirichlet distribution over the simplex whose concentration parameter is the all-ones vector plus t times the one-hot vector at the data corner e_i.

Now replace the straight line with a Dirichlet family

Replace the straight line with a family of Dirichlet distributions:

$$\boxed{p_t(x \mid x_1 = e_i) = \mathrm{Dir}\big(x;\ \alpha = \mathbf{1} + t\, e_i\big)}$$

The conditional path at time t is the Dirichlet distribution over the simplex whose concentration parameter is the all-ones vector plus t times the one-hot vector at the data corner e_i.

The i-th concentration is 1 + t, while the others remain at 1. As t increases, we place more probability near the data corner.

Now replace the straight line with a Dirichlet family

Replace the straight line with a family of Dirichlet distributions:

$$\boxed{p_t(x \mid x_1 = e_i) = \mathrm{Dir}\big(x;\ \alpha = \mathbf{1} + t\, e_i\big)}$$

The conditional path at time t is the Dirichlet distribution over the simplex whose concentration parameter is the all-ones vector plus t times the one-hot vector at the data corner e_i.

The i-th concentration is 1 + t, while the others remain at 1. As t increases, we place more probability near the data corner.

At t = 0, we start generation from the uniform distribution on the simplex.

Substitute that concentration into the density

Substitute that concentration into the density

Substitute $\alpha = 1 + t\, e_i$ into the density:

$$p_t(x \mid x_1 = e_i) = \frac{\Gamma(t+K)}{\Gamma(t+1)}\, x_i^t$$

Substitute that concentration into the density

Substitute $\alpha = 1 + t\, e_i$ into the density:

$$p_t(x \mid x_1 = e_i) = \frac{\Gamma(t+K)}{\Gamma(t+1)}\, x_i^t$$

The conditional density equals $\Gamma(t + K)$ divided by $\Gamma(t + 1)$, times the data coordinate x_i raised to the power t.

Substitute that concentration into the density

Substitute $\alpha = 1 + t\, e_i$ into the density:

$$p_t(x \mid x_1 = e_i) = \frac{\Gamma(t+K)}{\Gamma(t+1)}\, x_i^t$$

The conditional density equals $\Gamma(t + K)$ divided by $\Gamma(t + 1)$, times the data coordinate x_i raised to the power t.

Only the i-th exponent $\alpha_k - 1$ is nonzero, with value t. The beta function reduces to $\Gamma(t + 1)$ divided by $\Gamma(t + K)$.

Substitute that concentration into the density

Substitute $\alpha = 1 + t\, e_i$ into the density:

$$p_t(x \mid x_1 = e_i) = \frac{\Gamma(t+K)}{\Gamma(t+1)}\, x_i^t$$

The conditional density equals $\Gamma(t + K)$ divided by $\Gamma(t + 1)$, times the data coordinate x_i raised to the power t.

Only the i-th exponent $\alpha_k - 1$ is nonzero, with value t. The beta function reduces to $\Gamma(t + 1)$ divided by $\Gamma(t + K)$.

At $t = 0$ the density is the constant $\Gamma(K)$, which is the uniform distribution on the simplex. For DNA that constant is 6.

Substitute that concentration into the density

Substitute $\alpha = 1 + t\, e_i$ into the density:

$$p_t(x \mid x_1 = e_i) = \frac{\Gamma(t+K)}{\Gamma(t+1)}\, x_i^t$$

The conditional density equals $\Gamma(t + K)$ divided by $\Gamma(t + 1)$, times the data coordinate x_i raised to the power t.

Only the i-th exponent $\alpha_k - 1$ is nonzero, with value t. The beta function reduces to $\Gamma(t + 1)$ divided by $\Gamma(t + K)$.

At $t = 0$ the density is the constant $\Gamma(K)$, which is the uniform distribution on the simplex. For DNA that constant is 6.

For every finite t the density is positive wherever $x_i > 0$, so no corner is ever ruled out.

Substitute that concentration into the density

Substitute $\alpha = 1 + t\, e_i$ into the density:

$$p_t(x \mid x_1 = e_i) = \frac{\Gamma(t+K)}{\Gamma(t+1)}\, x_i^t$$

The conditional density equals $\Gamma(t + K)$ divided by $\Gamma(t + 1)$, times the data coordinate x_i raised to the power t.

Only the i-th exponent $\alpha_k - 1$ is nonzero, with value t. The beta function reduces to $\Gamma(t + 1)$ divided by $\Gamma(t + K)$.

At $t = 0$ the density is the constant $\Gamma(K)$, which is the uniform distribution on the simplex. For DNA that constant is 6.

Every corner remains possible at finite time, so a small change in the state need not rule out a token abruptly.

Time runs over the whole half-line

Time runs over the whole half-line

The Dirichlet concentration 1 + t reaches a point mass at the corner only as t grows without bound.

Time runs over the whole half-line

The Dirichlet concentration 1 + t reaches a point mass at the corner only as t grows without bound.

$$t \in [0, \infty), \qquad t \sim \mathrm{Exp}(1) \text{ in training}, \qquad t_{\max} = 8 \text{ at inference}$$

Time runs over the whole half-line

The Dirichlet concentration 1 + t reaches a point mass at the corner only as t grows without bound.

$$t \in [0, \infty), \qquad t \sim \mathrm{Exp}(1) \text{ in training}, \qquad t_{\max} = 8 \text{ at inference}$$

Time ranges over the nonnegative half-line. During training, we draw t from an exponential distribution with rate one. During inference, we integrate up to t = 8.

Time runs over the whole half-line

The Dirichlet concentration 1 + t reaches a point mass at the corner only as t grows without bound.

$$t \in [0, \infty), \qquad t \sim \mathrm{Exp}(1) \text{ in training}, \qquad t_{\max} = 8 \text{ at inference}$$

Time ranges over the nonnegative half-line. During training, we draw t from an exponential distribution with rate one. During inference, we integrate up to t = 8.

We still use x_1 for the data sample. Its subscript no longer denotes time one.

Time runs over the whole half-line

The Dirichlet concentration 1 + t reaches a point mass at the corner only as t grows without bound.

$$t \in [0, \infty), \qquad t \sim \mathrm{Exp}(1) \text{ in training}, \qquad t_{\max} = 8 \text{ at inference}$$

Time ranges over the nonnegative half-line. During training, we draw t from an exponential distribution with rate one. During inference, we integrate up to t = 8.

We still use x_1 for the data sample. Its subscript no longer denotes time one.

In Dirichlet Flow Matching we take 100 Euler steps up to t = 8 and then read the token off the final posterior.

Next, we need the velocity for this path

Next, we need the velocity for this path

Keep the direction and rescale the speed:

$$\boxed{u_t(x \mid x_1 = e_i) = C(x_i, t)\,(e_i - x)}$$

Next, we need the velocity for this path

Keep the direction and rescale the speed:

$$u_t(x \mid x_1 = e_i) = C(x_i, t)\,(e_i - x)$$

The conditional velocity at x equals a scalar speed $C(x_i, t)$, which depends on the data coordinate and the time, times the direction e_i – x from the current point to the data corner.

Next, we need the velocity for this path

Keep the direction and rescale the speed:

$$u_t(x \mid x_1 = e_i) = C(x_i, t)\,(e_i - x)$$

The conditional velocity at x equals a scalar speed $C(x_i, t)$, which depends on the data coordinate and the time, times the direction e_i – x from the current point to the data corner.

The straight-line path is the special case C = 1/(1 – t), a function of time alone.

Next, we need the velocity for this path

Keep the direction and rescale the speed:

$$u_t(x \mid x_1 = e_i) = C(x_i, t)\,(e_i - x)$$

The conditional velocity at x equals a scalar speed $C(x_i, t)$, which depends on the data coordinate and the time, times the direction $e_i - x$ from the current point to the data corner.

The straight-line path is the special case $C = 1/(1 - t)$, a function of time alone.

The direction $e_i - x$ has coordinates summing to zero, so the velocity is tangent to the simplex and the point stays on it.